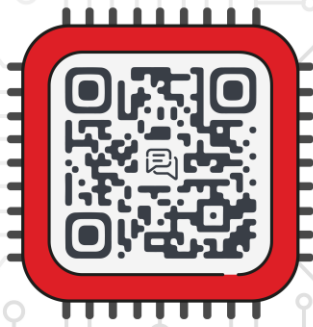




5

71

10650 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 27/11/2025

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

التدريب 2

(Backpropagation)

عملي مشترك



الشبكات العصبونية

تذكرة :

الفرق بين Gradient و Derivative

الفرق بين المشتق والتدرج هو فرق في البعد (الأبعاد التي نعمل بها)، وهذا الفرق يغير أليتهما وأهميتهما بشكل جذري.

لنسط الأمر، تخيل أنك تقف على تلة:

المشتق (Derivative):

- تخيل أنك على طريق مستقيم واحد يمتد صعوداً وهبوطاً (بُعد واحد).
- المشتق هو مقدار انحدار الطريق في النقطة التي تقف فيها بالضبط.
- هل هو حاد (+5) أم منحدر للأسفل (-3)؟
- إنه مجرد رقم (قيمة عددية) يخبرك عن معدل التغير.

التدرج (Gradient):

- تخيل الآن أنك في وسط حقل به تلال وأودية من كل جانب (أبعاد متعددة).
- التدرج ليس مجرد رقم، بل هو مثل بوصلة تشير إلى الاتجاه الذي يصعد فيه المرتفع بأسرع شكل ممكن من مكانك الحالي.
- هو متجه (vector) له اتجاه وقيمة

مثال توضيحي للآلية:

لدينا الدالة $f(x, y) = x^2 + y^3$

$$\frac{\partial f}{\partial x} = 2x$$

المشتق الجزئي بالنسبة لـ x نعتبر y ثابت :

$$\frac{\partial f}{\partial y} = 3y^2$$

المشتق الجزئي بالنسبة لـ y نعتبر x ثابتة :

إذا التدرج (Gradient) هو:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (2x, 3y^2)$$

لاحظ أن الناتج متجه، بينما لو كانت الدالة $f(x) = x^2$ فقط، لكان المشتق $f'(x) = 2x$ وهو قيمة عددية.

وجه المقارنة	المشتق	التدرج
الدالة	دالة بمتغير واحد	دالة بأكثر من متغير $f(x,y,...)$
الناتج	قيمة عددية (scalar)	متجه (vector)
المعنى الهندسي	ميل المماس المنحني	متجه يشير إلى أسرع صعود إلى السطح
الاستخدام الرئيسي	تحليل التغير في بعد واحد	التحسن والبحث عن الحلول المثلى في أبعاد متعددة

خلاصة القول :

التدرج هو تعميم للمشتق في الأبعاد المتعددة.

إذا فهمت المشتق على أنه معدل التغير في خط واحد، فإن التدرج هو معدل التغير وموجهته في فضاء متعدد الأبعاد. المشتق يعطيك كم يتغير الشيء، بينما التدرج يعطيك كم و إلى أين يتغير الشيء بأسرع ما يمكن

Gradient Descent

لنستخدم تشبيه بسيط للتذكير بالفكرة :

النزول من جبل في ليلة مظلمة

تخيل أنك واقف على جبل في منتصف الليل، وهدفك هو الوصول إلى أدنى نقطة في الوادي ، المشكلة أنك لا ترى شيئاً، لا تعرف أين يقع الوادي بالضبط، ولا يمكنك رؤية الخريطة الكاملة للجبل.

ماذا ستفعل؟

ستفعل ما يفعله هبوط التدرج بالضبط:

■ **تشعر بقدمك :** ستلتف حولك وتشعر بالأرض تحت قدميك لمعرفة الاتجاه الذي ينحدر فيه الجبل بأكثر حدة .

هذا الاتجاه هو التدرج

■ **تخطو خطوة صغيرة نحو الأسفل:** بما أن الهدف هو الوصول للوادي، ستأخذ خطوة صغيرة في الاتجاه

المعاكس لل Gradient (الانحدار) (أي أنك تختار الطريق الذي ينزل بك لأسفل بدلاً من الذي يصعد بك)

■ **تكرر العملية :** في موقعك الجديد، ستكرر العملية مرة أخرى: تشعر بالانحدار، وتتخذ خطوة صغيرة في الاتجاه المعاكس له.

■ **تستمر :** تستمر في فعل هذا، خطوة بخطوة، حتى تشعر أن الأرض من حولك مستوية تقريباً في كل

الاتجاهات. عندها تعرف أنك وصلت إلى أدنى نقطة في الوادي (أو على الأقل إلى نقطة منخفضة).

كيف يرتبط هذا في تعلم الآلة؟

- في تعلم الآلة، الجبل هو دالة الخسارة (Loss Function)
- دالة الخسارة: هي معادلة تقيس مدى خطأ أو سوء أداء النموذج (كلما كان النموذج أسوأ في أداء مهمته، كانت قيمة الخسارة أعلى (أي أننا في مكان مرتفع على الجبل)).
- موقعك على الجبل: يمثل أوزان النموذج (Parameters/Weights). هذه هي الأرقام التي يحاول النموذج تعلمها ليعمل بشكل صحيح.
- الهدف: إيجاد الأوزان المثالية التي تجعل دالة الخسارة في أدنى نقطة ممكنة (الوصول إلى قاع الوادي). هذا يعني أن النموذج أصبح دقيقاً قدر الإمكان وأقل خطأ.

إذاً، هبوط التدرج هو الخوارزمية التي تقوم بالآتي :

- تبدأ بأوزان عشوائية (موقع عشوائي على الجبل)
- حساب الميل (التدرج): تحسب في أي اتجاه تزيد الخسارة
- تعديل الأوزان (الخطوة): تقوم بتعديل الأوزان بالتحرك في عكس اتجاه الميل (أي باتجاه تقليل الخطأ)
- تكرر هذه العملية آلاف أو ملايين المرات، كل مرة تقلل الخطأ قليلاً، حتى تصل إلى أدنى نقطة ممكنة للخطأ.

المكونات الأساسية:

- التدرج (Gradient): البوصلة التي تشير إلى اتجاه الزيادة الأكبر في الخطأ.
- معدل التعلم (Learning Rate): حجم خطواتك. إذا كانت خطواتك كبيرة جداً، قد تتخطى الوادي. إذا كانت صغيرة جداً، سيستغرقك وقت طويل جداً للوصول إليه. اختيار حجم الخطوة المناسب هو فن بحد ذاته.

الخلاصة :

هبوط التدرج هو المحرك الأساسي الذي تتعلم به معظم النماذج. إنه ببساطة عملية تكرارية لتصحيح الأخطاء، خطوة بخطوة، في الاتجاه الصحيح، حتى يصل النموذج إلى أفضل أداء ممكن .

والآن بالعودة لمحاضرتنا الاساسية :

- لجعل الشبكة العصبية تتعلم وتحسن، يجب أن نعرف كيف نعدّل الإعدادات الداخلية لها، والتي تسمى الأوزان (weights).
- الهدف هو تعديل هذه الأوزان لتقليل أخطاء الشبكة .
- عملية حساب مقدار التعديل المطلوب لكل وزن تسمى حساب التدرج (gradient).

في البداية :

- كانت هناك طريقة بسيطة لحساب هذا التدرج، كانت التفاضل العددي (Numerical Differentiation) لكنها كانت بطيئة جداً وغير عملية .
- تخيل أن لديك شبكة عصبية ضخمة تحتوي على ملايين الأوزان، والطريقة القديمة تتطلب منك فحص كل وزن على حدة لمعرفة تأثيره، وهو ما يشبه محاولة ضبط كل برغي في طائرة ضخمة بمفتاح واحد في كل مرة.
- تنويه : سنستخدم بدلا من كلمة مشتق كلمة تأثير لتعبير افضل وواضح

هنا يأتي دور الانتشار الخلفي (Backpropagation) :

- وهو خوارزمية ذكية وحديثة تحل هذه المشكلة. بدلاً من فحص كل وزن بشكل منفصل، يقوم الانتشار الخلفي بحساب تدرجات جميع الأوزان في الشبكة دفعة واحدة وبكفاءة عالية جداً
- لهذا السبب، أصبح هو الطريقة المعتمدة لتدريب معظم الشبكات العصبية اليوم.

لكي نفهم كيف تعمل هذه الخوارزمية هناك طريقتان:

1. طريقة المعادلات الرياضية

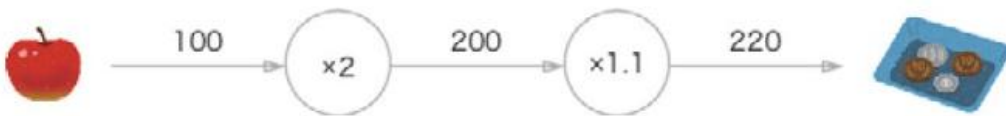
تعتمد على سلسلة من الصيغ المعقدة. ورغم أنها دقيقة، إلا أنها قد تكون صعبة الفهم.

2. طريقة المخططات الحسابية (Computational Graphs) :

- وهي طريقة بصرية وأكثر سهولة
- المخطط الحسابي: هو رسم بياني يفكك المعادلة الصعبة إلى عمليات صغيرة (مثل الضرب أو الجمع).
- الهدف: بدلاً من التعامل مع المسألة ككتلة واحدة، نراها كسلسلة من الخطوات السهلة وراء بعضها البعض
- مكوناته:
- عقد (Nodes): تمثل عمليات حسابية (مثل الضرب $\times$ أو الجمع $+$) أو متغيرات وتُرسم عادة على شكل دوائر.
- أسهم (Edges): تمثل تدفق البيانات أو النتائج من عملية إلى أخرى.

■ مثال 1: شراء تفاحتين

اشترى تارو تفاحتين بـ 100 ين لكل تفاحة. احسب المبلغ الإجمالي بعد إضافة ضريبة استهلاك 10% :
الحل : يمكن تمثيل هذا الحساب بالرسم البياني التالي

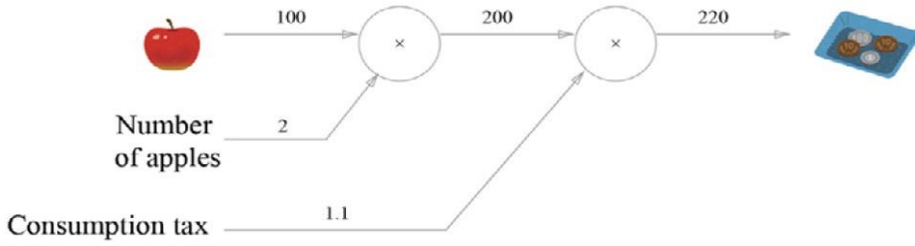
شرح الرسم:

الحساب الطبيعي من اليسار إلى اليمين :
 سعر التفاحة (100) يدخل إلى عقدة ($\times 2$) لتصبح 200
 الناتج (200) يدخل إلى عقدة ($\times 1.1$) لينتج المبلغ النهائي 220

طريقة أخرى للرسم (لزيادة المرونة) :

- بدلاً من وضع الأرقام (2 و 1.1) داخل دوائر العمليات، يمكننا وضعها خارجها كمتغيرات.
- الدائرة تحتوي فقط على رمز العملية ($\times$)
- الأرقام مثل عدد التفاحات أو قيمة الضريبة توضع كمتغيرات خارجية

كما في الشكل التالي :



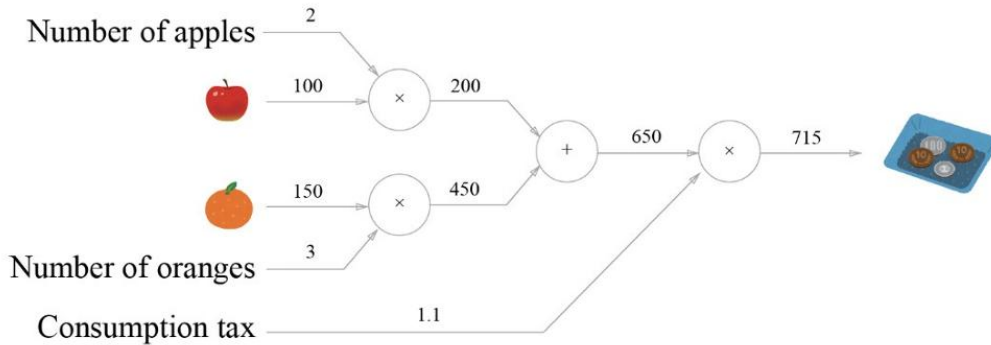
لماذا هذه الطريقة أفضل؟

- لأنها تجعل المخطط أكثر مرونة.
- الآن، إذا أردنا حساب سعر 3 تفاحات بدلاً من 2، أو إذا تغيرت نسبة الضريبة، كل ما علينا فعله هو تغيير قيمة المتغير الخارجي دون تغيير بنية المخطط نفسه.
- هذا المبدأ هو أساس عمل الشبكات العصبية، حيث يمكن تغيير المدخلات (الأوزان) وإعادة الحساب بسهولة

■ مثال 2 :

اشترى تارو تفاحتين وثلاث برتقالات . كانت التفاحة بسعر 100 ين، والبرتقالة بسعر 150 ين . تم تطبيق ضريبة استهلاك بنسبة 10% احسب المبلغ الذي دفعه.

الحل : يمكن تمثيل هذا الحساب بالرسم البياني التالي:



- لجمع تكلفة التفاح مع تكلفة البرتقال، نستخدم عقدة الجمع (+)
- هذه العقدة بسيطة جداً: هي تأخذ مبلغين من المال (وهما عبارة عن عدد التفاحات (2) مضروبين ب سعر التفاحة الواحدة (100) و عدد البرتقالات (3) مضروبين أيضا بسعر البرتقالة الواحدة (150)) وتقوم بجمعهما معاً لتمرر المجموع إلى الخطوة التالية. هذا يسمح لنا بجمع التكاليف الجزئية قبل حساب الضريبة الإجمالية

- بعد الجمع نقوم بضرب الناتج من خلال العقدة (x) ب الضريبة 1.1 ليصبح الناتج النهائي 715 ين

كيفية سير الحساب :

- بمجرد أن نرسم المخطط الحسابي الكامل الذي يمثل المشكلة، تبدأ عملية الحساب الفعلية.
- هذه العملية تسير من اليسار إلى اليمين بشكل منهجي ومنظم.

- تخيل أن الأرقام تتدفق عبر المخطط تمامًا كما يتدفق التيار الكهربائي في دائرة كهربائية.
- كل عقدة (دائرة) تقوم بعملها الحسابي (ضرب أو جمع) وتمرر الناتج إلى العقدة التي تليها على يمينها.
- تستمر هذه العملية حتى نصل إلى النهاية اليمنى من المخطط، حيث نحصل على الجواب النهائي للمسألة.

الخطوات الأساسية للحل:

لحل أي مسألة باستخدام هذه المخططات الحسابية، نتبع دائمًا خطوتين رئيسيتين:

- تصميم المخطط: أولاً، يجب عليك تحليل المسألة وترجمة خطواتها إلى رسم بياني من العقد والأسهم. هذا أشبه بوضع خريطة أو تصميم للحل.
- تنفيذ الحساب: ثانياً، تبدأ بالقيم الأولية على الجانب الأيسر وتدعها تتدفق عبر المخطط خطوة بخطوة حتى تصل إلى الناتج النهائي على الجانب الأيمن .

بعد أن رسمنا خريطة الحساب (المخطط الحسابي)، نحتاج إلى طريقة لاستخدامها للحصول على النتيجة. هنا يأتي مفهومان مهمان: الانتشار الأمامي والانتشار الخلفي

1. الانتشار الأمامي (Forward Propagation)

هذا هو المسار الطبيعي والبدهي للحساب. تخيل أن الأرقام والبيانات تتحرك على طول خط تجميع في مصنع.

- كيف يعمل؟ تبدأ بالقيم الأولية في الطرف الأيسر من المخطط (مثل سعر التفاحة وعددها).
- تمر هذه القيم عبر كل محطة عمل (عقدة) على طول الخط. كل محطة تقوم بعملها المخصص (مثل الضرب أو الجمع).
- تستمر العملية حتى نصل إلى نهاية خط التجميع في الطرف الأيمن، حيث نحصل على المنتج النهائي (الإجابة النهائية للمسألة).

هذه العملية التي تسير من البداية إلى النهاية، من اليسار إلى اليمين، تسمى الانتشار الأمامي .

2. الانتشار الخلفي (Backward Propagation)

إذا كان هناك مسار طبيعي من اليسار إلى اليمين، فهل يمكننا السير في الاتجاه المعاكس؟ نعم، هذا ممكن وهو مهم جداً.

- كيف يعمل؟ تخيل أنك تبدأ من النتيجة النهائية في الطرف الأيمن وترجع خطوة بخطوة إلى الخلف، من اليمين إلى اليسار.
- ما هو الهدف؟ هذا المسار العكسي ليس للحصول على النتيجة النهائية ، بل لفهم كيف ساهمت كل قيمة أولية في الوصول إلى هذه النتيجة.

بعبارة أخرى، هو يساعدنا على حساب المشتق لكل متغير.

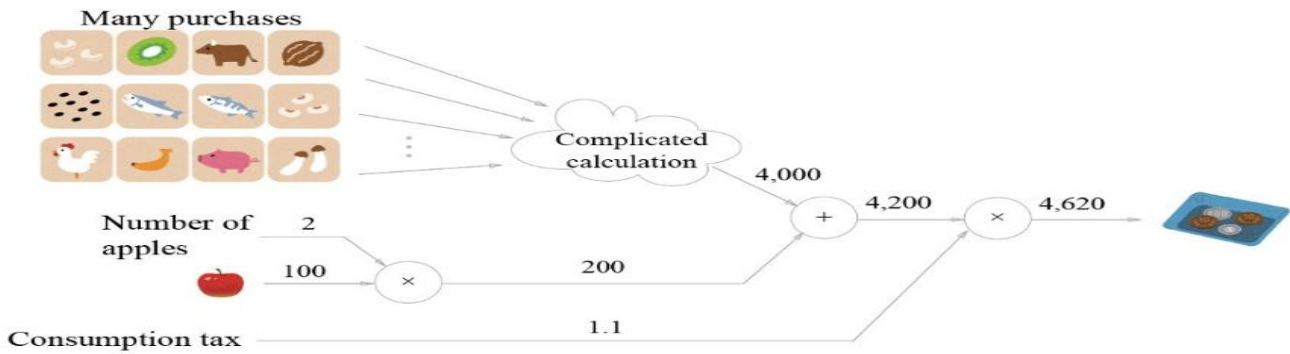
هذه العملية العكسية تسمى الانتشار الخلفي أو الارتداد الخلفي (Backpropagation)، وهي حجر الزاوية في تدريب الشبكات العصبية لأنها تخبرنا بكيفية تعديل الأوزان لتحسين الأداء .

الميزة الرئيسية: الحساب الموضعي (Local Calculation)

- كل عقدة تؤدي مهمتها البسيطة فقط، دون الحاجة إلى معرفة الصورة الكاملة للمشكلة هذا يسمح لنا بتقسيم أي حساب معقد إلى خطوات بسيطة ومستقلة.
- أي : كل عقدة (node) يهتمها فقط قيمة الدخل ولا يهتمها كيف يتم حساب قيمة الدخل لكي تعطي الناتج

مثال:

تخيل أنك عند صندوق الدفع في السوبر ماركت.



- اشتريت الكثير من الأشياء المختلفة، وبعد عملية حسابية معقدة، بلغ إجمالي فاتورة المشتريات 4000 ين
- بعد ذلك، تذكرت أن لديك تفاحتين لم يتم تسعيرهما بعد، وسعرهما 200 ين
- الآن، مهمة موظف الصندوق التالية هي جمع المبلغين $4000 + 200 = 4200$

انتبه للنقطة المهمة هنا:

- موظف الصندوق لا يحتاج إلى معرفة كيف تم حساب الـ 4000 ين. هل كانت من 10 منتجات أم 50؟ هل كان هناك خصومات؟ كل هذا لا يهمه.
- مهمته الموضعية هي فقط أن يأخذ الرقمين اللذين أمامه (4000 و 200) ويقوم بعملية واحدة بسيطة وهي الجمع.

هذا هو الحساب الموضعي كل عقدة في مخططنا تعمل مثل موظف الصندوق هذا :

- هي تهتم فقط بالبيانات التي تستقبلها مباشرة.
- تقوم بعملية حسابية واحدة بسيطة (ضرب، جمع، إلخ).
- تمرر الناتج إلى العقدة التالية.

هذه الخاصية تسمح لنا بتقسيم أي مشكلة حسابية معقدة إلى سلسلة من الخطوات البسيطة والمنفصلة، مما يجعل حلها وتتبّعها (سواء للأمام أو للخلف) أمرًا ممكنًا ومنظمًا .

الخلاصة :

الهدف هو أن تتقن التفكير في أي عملية حسابية على أنها سلسلة من الخطوات البسيطة التي يمكن تمثيلها في مخطط. هذا التفكير هو المفتاح لفهم كيفية عمل الانتشار الخلفي (Backpropagation) الذي سنستخدمه لاحقاً لحساب التدرجات في الشبكات العصبية .

■ بعد أن تعلمنا كيفية رسم واستخدام المخططات الحسابية، يمكننا أن نسأل سؤالاً مهماً: لماذا كل هذا العناء؟ ما هي الفائدة الحقيقية من استخدام هذه المخططات بدلاً من مجرد كتابة المعادلات وحلها مباشرة؟

الجواب يكمن في ثلاث مزايا رئيسية:**1. الميزة الأولى: تبسيط المشاكل المعقدة (الحساب الموضعي)**

كما رأينا سابقاً، الميزة الأولى هي أن المخطط يقسم أي مشكلة معقدة إلى خطوات صغيرة وبسيطة جداً. كل عقدة (دائرة) في المخطط تهتم بمهمتها فقط، تماماً مثل كل عامل في خط تجميع للسيارات يهتم بتركيب قطعة واحدة فقط. هذا يسهل فهم المشكلة وتصحيح الأخطاء إذا حدثت.

2. الميزة الثانية: الحفاظ على سجل الحسابات

عندما نحل مسألة باستخدام مخطط حسابي، فإننا نحتفظ بجميع النتائج الوسيطة. على سبيل المثال، في مسألة التفاح والبرتقال، نحن نعرف أن سعر التفاح بعد الضرب كان 200 ين، وسعر البرتقال كان 450 ين، ومجموعهما قبل الضريبة كان 650 ين.

هذا السجل مفيد لأنه يسمح لنا بالعودة والتحقق من أي خطوة في أي وقت.

3. الميزة الثالثة والأهم: حساب التأثير بكفاءة

هنا تكمن القوة الحقيقية للمخططات الحسابية، وهي السبب الرئيسي لاستخدامها في تعلم الآلة. لنعود إلى مثال التفاحتين. لقد حسبنا أن الفاتورة النهائية هي 220 ين. لكن، ماذا لو أردنا أن نعرف شيئاً أعمق؟ ماذا لو أردنا أن نعرف مدى تأثير سعر التفاحة الواحدة على الفاتورة النهائية؟ بعبارة أخرى: إذا زادت سعر التفاحة بمقدار 1 ين، فكم سيزداد المبلغ الإجمالي الذي ندفعه؟ هذا التأثير يسمى رياضياً المشتق (Derivative) معرفة هذا التأثير ضروري جداً إذا أردنا تحسين أي نظام.

■ كيف تساعدنا المخططات الحسابية هنا؟

■ بدلاً من إعادة حل كل المعادلة من البداية لكل تغيير بسيط في السعر، يمكننا استخدام نفس المخطط، ولكن هذه المرة بالعكس!

■ سنبدأ من النتيجة النهائية (220 ين) وننتشر للخلف، من اليمين إلى اليسار. في كل عقدة، سنحسب التأثير المحلي لتلك العقدة ونمرره للخطوة التي تسبقها. هذه العملية تسمى الانتشار الخلفي (Backpropagation)

■ النتيجة:

من خلال هذا الانتشار الخلفي، يمكننا حساب تأثير كل متغير في المسألة (سعر التفاحة، عدد التفاح، نسبة الضريبة) على النتيجة النهائية في خطوة واحدة فقط وبدقة عالية. هذا الأمر مستحيل تقريباً مع الطرق التقليدية في المسائل المعقدة.

■ الخلاصة:

نستخدم المخططات الحاسوبية لأنها تجعل الحسابات المعقدة بسيطة، وتحفظ لنا جميع الخطوات الوسيطة، والأهم من ذلك كله، تسمح لنا بحساب تأثير كل جزء من المعادلة على النتيجة النهائية بكفاءة هائلة عبر الانتشار الخلفي. هذه القدرة هي حجر الزاوية في تدريب الشبكات العصبية الحديثة.

كيف يعمل الانتشار الخلفي خطوة بخطوة؟

تخيل أنك مدير في شركة وربح الشركة انخفض هذا الشهر أنت لا تعرف السبب، هل المشكلة في مواد الخام؟ أم في فريق المبيعات؟ أم في الإعلانات؟

الشركة تعمل كخط تجميع، تماماً مثل المخطط الحسابي:

■ قسم المواد الخام يشتري المواد.

■ قسم الإنتاج يصنع المنتجات.

■ قسم المبيعات يبيع المنتجات.

■ في النهاية نحصل على الربح النهائي.

هذا هو الانتشار الأمامي (Forward Propagation) : سير العمل الطبيعي من البداية إلى النهاية.

الآن لماذا نحتاج إلى الانتشار الخلفي؟

لأن الربح انخفض، وأنت تريد معرفة السبب الحقيقي لكي تقوم بإصلاحه. لا يمكنك أن تلوم الجميع، بل تريد أن تعرف مدى تأثير كل قسم على هذا الانخفاض.

هنا تبدأ التحقيق، وهذا هو الانتشار الخلفي (Backpropagation) ستبدأ التحقيق من النهاية وترجع خطوة بخطوة للخلف.

التحقيق (الانتشار الخلفي خطوة بخطوة):

■ تبدأ من النتيجة النهائية :

تقول: الربح انخفض بمقدار 10,000 دولار (هذه هي نقطة بدايتك)

■ تذهب إلى قسم المبيعات:

تسأل مدير المبيعات: الربح انخفض 10 آلاف، ما هو نصيب المبيعات من هذا الانخفاض؟

قد يجيبك: انخفاض مبيعاتنا هو المسؤول عن 80% من المشكلة

إذا تأثير قسم المبيعات هو 80% من 10,000 أي 8000 دولار .

■ تذهب إلى قسم الإنتاج :

تسأله: الربح انخفض 10 آلاف، والمبيعات مسؤولة عن 8 آلاف، إذاً يتبقى 2000 دولار. ما هو سبب هذه الزيادة في

التكاليف؟

قد يجيبك: ارتفاع سعر المواد الخام هو المسؤول عن 50% من هذه الزيادة

إذاً تأثير سعر المواد الخام هو 50% من 2000 أي 1000 دولار .

■ تذهب إلى قسم الإعلانات :

تسأله: المبيعات انخفضت وتأثيرها 8 آلاف دولار، ما هو سبب هذا الانخفاض؟
قد يجيبك: تخفيض ميزانية الإعلانات هو السبب الرئيسي، وهو مسؤول عن 60% من انخفاض المبيعات
إذاً تأثير ميزانية الإعلانات هو 60% من 8000 اي 4800 دولار .

ماذا استفدنا من هذا التحقيق؟

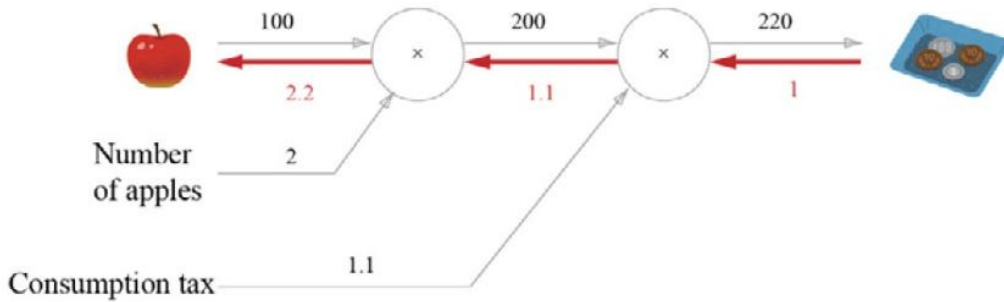
الآن لديك خريطة تأثير واضحة :

■ ميزانية الإعلانات كان لها التأثير الأكبر (-4,800 دولار).

■ سعر المواد الخام كان له تأثير أقل (-1,000 دولار).

الآن تعرف بالضبط أين تركز جهودك لإصلاح المشكلة.

لنعد الآن إلى مثال التفاحة : الانتشار الخلفي في مسألة التفاحة هو نفس الشيء تماماً، لكن بدلاً من الدولارات، نستخدم أرقاماً صغيرة



نعبّر عن الانتشار الخلفي بأشهر في الاتجاه المعاكس للانتشار الامامي وهي باللون الاحمر كما هو موضح بالمخطط

سنبداً من النهاية ونذهب للخلف اي من اليمين الى اليسار :

الخطوة 1: نقطة البداية (النهاية الفعلية للمسار اي اقصى اليمين)

- السؤال: ما هي تأثير (مشتق) المبلغ النهائي (L) للتغيير في نفسه أي بعبارة أخرى تأثير المبلغ النهائي (L) على نفسه ؟ (زد الرقم X بمقدار 1، فكم زاد الرقم X ؟)
- الإجابة: إذا زدت المبلغ النهائي بمقدار 1، فإنه يزداد بمقدار 1 (مشتق أي متغير بالنسبة لنفسه تساوي 1)
- تنويه: العلاقة بين أي شيء والتغيير في نفسه هي دائماً واحد لواحد .
- النتيجة: هذه هي إشارة البداية التي نطلقها في رحلتنا إلى الخلف القيمة هي 1 .

الخطوة 2: تتبع التأثير عبر عقدة الضريبة :

- **العلاقة:** $(1.1 \times \text{المبلغ قبل الضريبة} = \text{المبلغ النهائي})$ (أي عند كل عقدة، نأخذ القيمة القادمة من اليمين ونضربها في المشتق الموضعي لتلك العقدة)
- **السؤال:** الآن نريد أن نعرف: كيف ينتقل التأثير من المبلغ النهائي إلى المبلغ الذي قبله؟
- **المنطق:** لقد وجدنا أن تأثير المبلغ النهائي على نفسه هو 1 لكن المبلغ النهائي نفسه جاء من ضرب المبلغ الذي قبله في 1.1
- **إذاً** أي تأثير يصل إلى المبلغ النهائي، يجب أن يكون قد مر عبر عملية الضرب هذه
- **الحساب:** لنعرف تأثير المبلغ قبل الضريبة، نأخذ التأثير الذي وصل إلى النهاية (وهو 1) ونضربه في الرقم الذي تم به الضرب (وهو 1.1)
- **بعبارة أخرى:** عند عقدة $(1.1 \times)$ ، المشتق الموضعي هو 1.1 والقيمة القادمة من اليمين هي 1
- **إذاً:** $1.1 = 1.1 \times 1$
- **معنى النتيجة:** هذا يعني أن تأثير المبلغ قبل الضريبة على المبلغ النهائي هو 1.1
- **أو بعبارة أخرى:** أي تغيير في المبلغ قبل الضريبة سيظهر في المبلغ النهائي مضاعفاً 1.1 مرة ، هذه القيمة (1.1) هي الإشارة الجديدة التي نمررها إلى الخلف .

الخطوة 3: تتبع التأثير عبر عقدة التفاح

- **العلاقة:** المبلغ قبل الضريبة = سعر التفاحة $\times 2$
- **السؤال:** الآن، نريد أن نعرف تأثير سعر التفاحة
- **المنطق:** لقد وجدنا أن تأثير المبلغ قبل الضريبة على النهاية هو 1.1
- **الآن،** يجب أن نرى كيف ينتقل هذا التأثير إلى الورا أكثر، إلى سعر التفاحة. المبلغ قبل الضريبة نفسه جاء من ضرب سعر التفاحة في 2.
- **الحساب:** نأخذ التأثير الذي وصلنا إليه (وهو 1.1) ونضربه في الرقم الذي تم به الضرب في هذه الخطوة (وهو 2)
- **أي:** $2.2 = 1.1 \times 2$
- **معنى النتيجة النهائية:** هذا هو التأثير النهائي. يعني أن أي تغيير طفيف في سعر التفاحة سيؤثر على المبلغ النهائي بقوة 2.2 مرة .

ماذا يعني الرقم 2.2؟

هذا الرقم هو المشتق أو معدل التغير معناه ببساطة:
إذا زاد سعر تفاحة واحدة بمقدار 1 ين، فإن المبلغ الإجمالي الذي تدفعه سيزداد بمقدار 2.2 ين .

لماذا هذه الطريقة فعالة جداً؟

- الحصول على مشتقات متعددة في مرة واحدة: بنفس عملية التتبع الخلفي الواحدة، يمكننا حساب مشتق جميع المتغيرات .
- على سبيل المثال، لحساب مشتق عدد التفاح، نعود إلى عقدة (2 ×) ونستخدم المؤثر المحلي الآخر وهو سعر التفاحة (100). إذاً مشتق عدد التفاح سيكون $110 = 100 \times 1.1$ ومشتق الضريبة سيكون $200 = 200 \times 1$ كل هذا يتم في عملية واحدة.
- إعادة استخدام النتائج الوسيطة: لاحظ كيف أن القيمة الوسطية 1.1 تم حسابها مرة واحدة فقط، ثم استخدمت لإيجاد مشتق سعر التفاحة ومشتق عدد التفاح. هذا يوفر الكثير من الوقت والجهد مقارنة بحساب كل مشتق منفصل من البداية .

Chain Rule

لنشرح قاعدة السلسلة من خلال مثال :

تخيل أن لديك نظام صوت بسيط مكون من:

- ميكروفون هذا هو المدخل x
- مضخم صوت أول هذه الدالة الأولى f
- مضخم صوت ثاني هذه الدالة الثانية g
- سماعة هذا هو الخرج النهائي z

الصوت يمر بالترتيب التالي :

الميكروفون -> المضخم الأول -> المضخم الثاني -> السماعة

السؤال الأساسي:

إذا قمنا بزيادة صوت الميكروفون المدخل x بمقدار صغير جداً، فكم ستزداد شدة الصوت النهائي من السماعة الخرج z ؟

هذا هو بالضبط سؤال قاعدة السلسلة

لنحل المسار خطوة بخطوة:

الخطوة الأولى: المضخم الأول

لنفترض أن المضخم الأول يضاعف قوة الصوت الذي يستقبله أي أن تأثيره أو معامل التضخيم الخاص به هو 2

إذا زدنا صوت الميكروفون بمقدار 1، فإن خرج المضخم الأول سيزداد بمقدار 2 .

الخطوة الثانية: المضخم الثاني

لنفترض أن المضخم الثاني يضاعف قوة الصوت الذي يستقبله ثلاث مرات أي أن تأثيره أو معامل التضخيم الخاص به هو

3

إذا زدنا الصوت الداخل للمضخم الثاني بمقدار 1، فإن خرج (الصوت النهائي من السماعة) سيزداد بمقدار 3

ربط الخطوات معاً (السلسلة)

الآن، نريد أن نعرف التأثير الكلي من البداية إلى النهاية.

- نبدأ بزيادة صوت الميكروفون بمقدار 1

- هذا التغيير (1) يصل إلى المضخم الأول، فيقوم بتضخيمه ليصبح 2
- هذا التغيير (2) يصل الآن إلى المضخم الثاني، فيقوم هو الآخر بتضخيمه ثلاث مرات $2 \times 3 = 6$
- النتيجة النهائية: زيادة صوت الميكروفون بمقدار 1 أدت إلى زيادة الصوت النهائي بمقدار 6 .

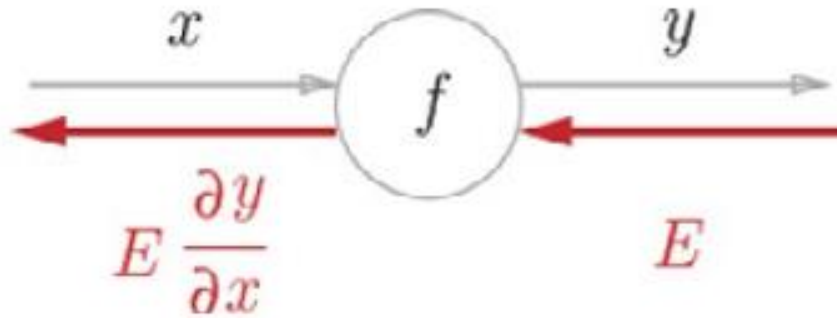
ماذا تعني هذه الأرقام؟

- تأثير المضخم الأول على الميكروفون (المشتق الموضعي الأول) : 2
 - تأثير المضخم الثاني على خرج الأول (المشتق الموضعي الثاني) : 3
 - التأثير الكلي للميكروفون على السماع (المشتق الكلي) : $2 \times 3 = 6$
- النظام الكامل للصوت هو الدالة المركبة (Composite Function) هي دالة كبيرة تتكون من دوال أصغر متصلة ببعضها.
- كل قسم من ضمن النظام على حدة يمثل دالة بسيطة.

مثال:

الدالة (المضخم الأول) : معادلتها $y = f(x)$

يوضح المخطط التالي الانتشار العكسي لهذه العملية الحسابية



لنفك العملية :

1. الإشارة القادمة من اليمين (E)

- تخيل أنك تقف عند مخرج هذا المضخم (y)
- من على يمينك، يأتي صوت (أو إشارة تأثير) قادم من باقي السلسلة.
- قوة هذا الصوت القادم هي E هذه الإشارة تمثل التأثير الذي وصل إلى هذه النقطة من كل المضخمات التي تليها
- وبشكل أوضح E هو مشتق تابع الخسارة بالنسبة ل y .

2. المشتق الموضعي (dy/dx)

- هذا هو معامل التضخيم الخاص بهذا المضخم فقط . إنه يخبرك كيف يغيّر هذا المضخم الصوت الذي يدخله.

- مثال: إذا كان المضخم هو x^2 ، y ، فإن معامل التضخيم (المشتق) هو $dy/dx = 2x$.

3. العملية:

- **السؤال هو:** ما هي قوة الصوت التي يجب أن تكون عند مدخل هذا المضخم (x) لتساهم في إنتاج هذا الصوت القادم (E) ؟
- **الإجابة هي:** أننا نأخذ الإشارة القادمة من اليمين (E) ونضربها في معامل التضخيم الخاص بالمضخم
- الإشارة الجديدة التي تمرر للخلف $E \times (dy/dx)$
- لنفترض أن المضخم هو $y = x^2$ إذا معامل التضخيم هو $2x$ ولنفترض أن الإشارة القادمة من اليمين (E) كانت قيمتها 10
- عند هذه العقدة، سنقوم بالعملية التالية: الإشارة الجديدة $= 2x = 2 * 10 = 20$

إذا القاعدة تنص على أن :

- عند كل عقدة (مضخم)، خذ الإشارة القادمة من اليمين واضربها في معامل التضخيم الخاص بالعقدة، ثم مرر الناتج إلى اليسار.
- عندما تفعل هذا لكل عقدة في السلسلة، من اليمين إلى اليسار، فإنك في النهاية تضرب جميع معاملات التضخيم معًا. وهذا هو بالضبط ما تفعله قاعدة السلسلة (Chain Rule) رياضياً .

ما هي الدالة المركبة (Composite Function) ؟

- تخيل أن لديك مصنعاً لإنتاج كعكة. هذا المصنع لا يقوم بكل شيء في خطوة واحدة. بل لديه مرحلتان:
- المرحلة الأولى (خلط المكونات): نأخذ المكونات الأولية الدقيق x والسكر y ونخلطها معًا لنحصل على خليط الكيك الذي سنسميه t .
- المعادلة: $t = x + y$
- المرحلة الثانية (الخبز): نأخذ خليط الكيك t ونضعه في الفرن، (فيتميز حجمها بطريقة ما) لينتج لنا الكعكة النهائية z
- المعادلة: $z = t^2$
- الدالة الكاملة $z = (x + y)^2$ هي دالة مركبة، لأنها تتكون من دالتين (مرحلتين) متصلتين ببعضهما البعض. مخرج المرحلة الأولى هو مدخل المرحلة الثانية.
- الآن، لنطرح أهم سؤال في المصنع :
- إذا قمنا بزيادة كمية الدقيق (x) بمقدار صغير جدًا، فكم ستتغير كمية الكعكة النهائية (z) ؟
- هذا السؤال معقد للإجابة عليه مباشرة. قاعدة السلسلة تمنحنا طريقة ذكية للإجابة عليه عن طريق تقسيم المشكلة.

فكرة قاعدة السلسلة ببساطة:

لنعرف التأثير الكلي من البداية إلى النهاية، يمكننا حساب التأثيرات الصغيرة لكل مرحلة على حدة ثم نضربها معاً

كأننا نسأل:

- كم تتغير الكعكة النهائية (z) إذا تغير خليط الكيك (t) ؟
- كم يتغير خليط الكيك (t) إذا تغير الدقيق (x) ؟
- التأثير الكلي = (تأثير المرحلة الثانية) × (تأثير المرحلة الأولى)

تطبيق القاعدة خطوة بخطوة :

لنطبق هذا على مثالنا:

■ الخطوة 1: حساب تأثير المرحلة الثانية (الخبز)

- السؤال: كيف تتغير الكعكة النهائية (z) بتغير الخليط (t) ؟
- المعادلة: $z = t^2$
- الإجابة (باستخدام قواعد الاشتقاق): إذا زدت t بمقدار 1، فإن z تزداد بمقدار $2t$
- هذا هو التأثير المحلي للمرحلة الثانية نكتبه كـ: $\frac{\partial z}{\partial t} = 2t$

■ الخطوة 2: حساب تأثير المرحلة الأولى (الخلط)

- السؤال: كيف يتغير خليط الكيك (t) بتغير الدقيق (x) ؟
- المعادلة: $t = x + y$
- الإجابة: إذا زدت x بمقدار 1، فإن t تزداد بنفس المقدار، وهو 1.
- هذا هو التأثير المحلي للمرحلة الأولى نكتبه كـ: $\frac{\partial t}{\partial x} = 1$

■ الخطوة 3: ضرب التأثيرات معاً (تطبيق قاعدة السلسلة) ،

- نجيب على سؤالنا الأصلي: التأثير الكلي للدقيق على الكعكة = (تأثير الخبز) × (تأثير الخلط)

أي :

$$\frac{\partial z}{\partial x} = \left(\frac{\partial z}{\partial t}\right) \times \left(\frac{\partial t}{\partial x}\right)$$

$$\frac{\partial z}{\partial x} = (2t) \times (1) = 2t$$

- الخطوة 4: إرجاع كل شيء إلى المتغيرات الأصلية تذكر أن t لم يكن سوى اسم مؤقت لـ $x + y$
- لنعود إلى المعادلة الأصلية :

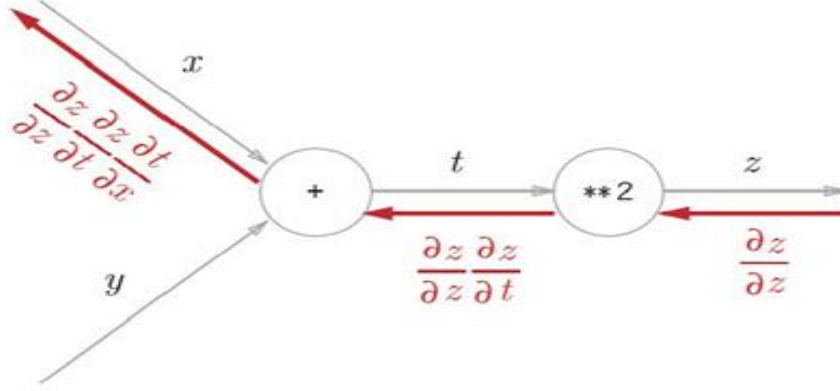
$$\frac{\partial z}{\partial x} = 2(x + y)$$

الخلاصة :

- قاعدة السلسلة هي استراتيجية فرق تسد
- بدلاً من محاولة حل معادلة معقدة $(x + y)^2$ مرة واحدة، قمنا بتقسيمها إلى جزأين أبسط بكثير

- وحسبنا تأثير كل جزء على حدة، ثم ضربنا النتائج لنحصل على الإجابة النهائية.
- هذا هو بالضبط ما يفعله الانتشار الخلفي (Backpropagation) في الشبكات العصبية: يمر عبر كل مرحلة (عقدة) ويحسب التأثير المحلي لها، ثم يضرب هذه التأثيرات معاً ليعرف التأثير الكلي لأي وزن في الشبكة.

والآن لنعبر عن هذا المثال من خلال Computational Graphs للتعبير عن حساب قاعدة السلسلة:



لدينا:

- مدخلات: x و y
- عملية أولى: جمع x و y
- عملية ثانية: تربيع نتيجة الجمع
- مخرج نهائي: z

الانتشار الخلفي (Backpropagation) هو عملية حساب المشتقات من المخرجات إلى المدخلات:

- يبدأ من المخرج z ويتحرك يساراً .
- في كل خطوة، يتم حساب المشتق الجزئي باستخدام قاعدة السلسلة .
- يتم ضرب المشتقات للحصول على المشتق النهائي .

كما نرى في المخطط : الغاية هي الوصول إلى مشتق z بالنسبة إلى x فالآلية كالتالي :

- الخطوة الاولى : بدأنا من اليمين : مشتق z بالنسبة ل نفسه هو $\frac{\partial z}{\partial z}$ ثم نمرر هذا الناتج الى اليسار
- الخطوة الثانية : وصلنا للعقدة التالية التي دخلها هي t : (اي حساب مشتق z بالنسبة ل t) اي سنأخذ الخرج من اليمين ونضربه بمشتق العقدة وينتج : $\frac{\partial z}{\partial t} \times \frac{\partial z}{\partial z}$
- الخطوة الثالثة : (حساب مشتق t بالنسبة ل x) نمرر الناتج الذي حصلنا عليه الى العقدة التي دخلها x فتكون الخطوة النهائية هي الناتج الذي من العقدة التي على اليمين مضروب ب مشتق العقدة الحالية فينتج :
- $\frac{\partial z}{\partial x} \times \frac{\partial z}{\partial t} \times \frac{\partial z}{\partial z}$

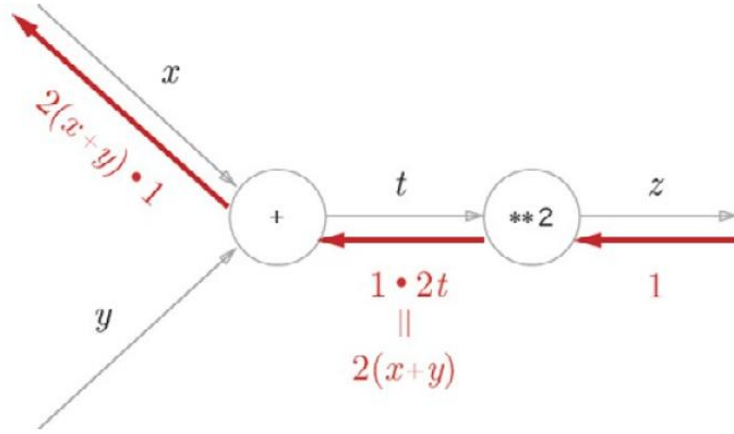
نقوم بالتعويض: $z = (x + y)^2 = t^2$ حيث $t = x + y$

مع الاخذ بعين الاعتبار أن : $\frac{\partial z}{\partial z} = 1$

$$\frac{\partial z}{\partial t} = 2t \times 1 = 2(x + y) \quad \blacksquare$$

$$\frac{\partial z}{\partial x} = 2(x + y) \times 1 \quad \blacksquare$$

لنرى النتيجة على المخطط :



تعميم :

قاعدة السلسلة هي أداة قوية تسمح لنا بحل مشاكل الاشتقاق المعقدة عن طريق:

1. تقسيم المشكلة إلى أجزاء بسيطة.
2. حساب التأثير المحلي لكل جزء.
3. ضرب هذه التأثيرات معاً للحصول على الإجابة النهائية.

هذا هو المبدأ الأساسي الذي تعمل عليه خوارزمية الانتشار الخلفي (Backpropagation) في الشبكات العصبية

الآن بعد أن فهمنا المبدأ العام لقاعدة السلسلة، حان الوقت لتعمق أكثر ونرى كيف تعمل هذه القاعدة داخل كل قطعة (عقدة) على حدة.

يوجد لدينا نوعان من العقد الجمع والضرب :

1. النوع الاول : عقدة الجمع (+)

مثال بسيط لتوضيح الفكرة :

فكر في عقدة الجمع (+) على أنها موصل رسائل بسيط جداً.

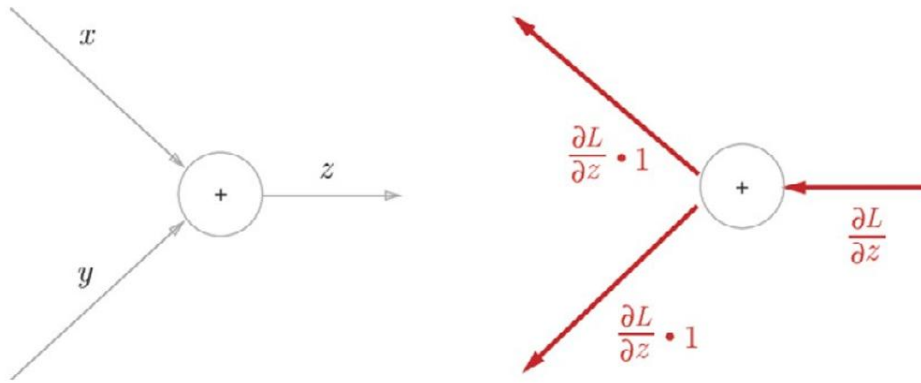
- هذا الموصل يستقبل رسالة واحدة مهمة من اليمين (هذه هي القيمة القادمة من بقية الشبكة، والتي نسميها

التدرج أو $\partial L / \partial z$ ومهمته هي توصيل هذه الرسالة إلى شخصين مختلفين على اليسار (المتغيرين x و y)

التعليمات : ما هي التعليمات التي أعطيت لهذا الموصل؟ التعليمات بسيطة جداً:

سلم الرسالة كما هي بالضبط , لا تغيّر فيها حرفاً واحداً

الآن، لنرى كيف يتوافق هذا المثال مع شبكتنا العصبية :



1. الرسالة الأصلية (الإشارة القادمة من اليمين): هذه هي قيمة المشتق القادمة من الطبقات اللاحقة في الشبكة. نرسم لها ب $\partial L / \partial z$ ، وهي تخبرنا بمدى أهمية المخرج z للخطأ النهائي L
2. قاعدة لا تغير شيئاً (المشتق المحلي local derivative): قاعدة سلم الرسالة كما هي رياضياً تعني ضرب في 1.

لهذا السبب، نرى في أن مشتق z بالنسبة لـ x هي $(\frac{\partial z}{\partial x} = 1)$

معنى $\frac{\partial z}{\partial x} = 1$: إذا زدت x بمقدار 1، فإن z تزداد بنفس المقدار، وهو 1

3. تسليم الرسائل (الحساب النهائي) :

عندما يصل الموصل إلى الشخص الأول (x) ، يسلمه الرسالة الأصلية مضروبة في 1 (أي كما هي) الرسالة المسلمة لـ $x = 1 \times$ (الرسالة الأصلية)

$$\frac{\partial L}{\partial x} = \left(\frac{\partial L}{\partial z} \right) \times \left(\frac{\partial z}{\partial x} \right) = \left(\frac{\partial L}{\partial z} \right) \times 1 = \frac{\partial L}{\partial z}$$

لدينا عملية حسابية طبيعية تسير من اليسار إلى اليمين:

■ العملية الأمامية (Forward): $10 + 5 = 15$

■ الانتشار الخلفي (السير عكس الاتجاه Backward) :

تخيل أن قيمة إشارة تصحيح أو مستوى أهمية قادمة من اليمين، وقيمتها هي 1.3 هذه الإشارة لا علاقة لها بالأرقام 10 أو 5، بل هي قيمة تأتي من مرحلة لاحقة في الشبكة لتخبرنا بمدى أهمية المخرج 15 هنا، عقدة الجمع (+) تعمل كمدير مشروع بسيط ومباشر .

ماذا يحدث؟

1. استلام الأمر :المدير (عقدة +) استلم أمراً من الإدارة العليا (اليمين) يقول: النتيجة النهائية (15) تحتاج إلى تعديل بقيمة 1.3

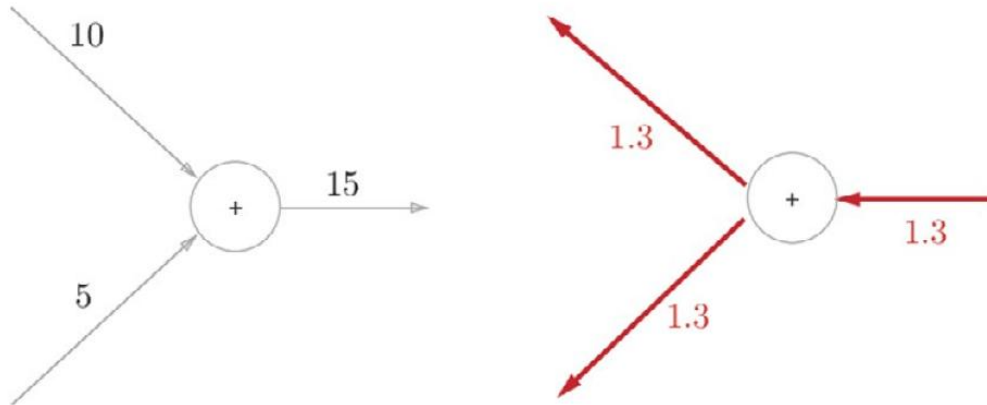
2. توزيع الأمر :ماذا سيفعل المدير؟ بما أن المساهمين في المشروع هما الرقم 10 والرقم 5، وبما أن مساهمتهم كانت عبر عملية جمع بسيطة، فإن المدير يوزع الأمر بشكل متساو وعادل.

3. النتيجة النهائية:

■ يذهب المدير إلى الرقم 10 ويقول له: مساهمتك تحتاج إلى تعديل بقيمة 1.3

■ ثم يذهب إلى الرقم 5 ويقول له نفس الشيء تماماً: مساهمتك تحتاج إلى تعديل بقيمة 1.3

لنرى المخطط للمثال



القاعدة الذهبية هنا هي :

عقدة الجمع (+) لا تغير في قيمة الإشارة، بل تكررها وتوزعها. هي مثل موزع كهربائي بسيط يأخذ تياراً معيناً ويخرجه من عدة مخارج بنفس القوة .

2. النوع الثاني : عقدة الضرب (x) :

مثال لتوضيح الفكرة :

نفترض أن نور وفاطمة شريكان في مشروع، ونجاح المشروع يعتمد على حاصل ضرب مجهودهما معاً (ليس مجموع المجهودين)
المرحلة الأولى (المرور الأمامي): إذا قدمت نور مجهوداً قيمته 10، وفاطمة مجهوداً قيمته 5، فإن نجاح المشروع سيكون
 $50 = 5 \times 10$

المرحلة الثانية (المرور الخلفي): وهي ان الإدارة طلبت تعديل النتيجة النهائية بمقدار بسيط (1.3 مثلاً). فكيف نوزع هذا التعديل بين نور وفاطمة ؟

بما أن العقدة هي ضرب، فإن تأثير مجهود نور يعتمد على قيمة مجهود فاطمة، والعكس صحيح.

فيكون نصيب نور من التعديل: نضرب (قيمة التعديل) $\times$ (مجهود فاطمة)

و نصيب فاطمة من التعديل: نضرب (قيمة التعديل) $\times$ (مجهود نور)

الخلاصة: في عقدة الضرب، كل طرف يأخذ حصته من التعديل بناءً على قوة الطرف الآخر. أي بمعنى تبديل الأدوار

لنعود إلى مثالنا:

■ مجهود نور $x = 10$

■ مجهود فاطمة $y = 5$

■ الملاحظة القادمة من اليمين 1.3

1. حصة نور من الملاحظة :

• حصة نور $= 1.3 \times \text{مجهود فاطمة}$

• حصة نور $= 1.3 \times 5 = 6.5$

2. حصة فاطمة من الملاحظة :

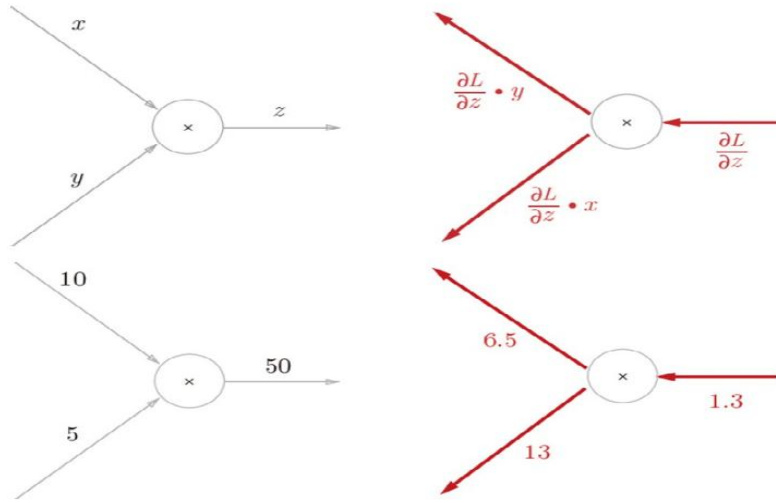
• حصة فاطمة = $1.3 \times$ مجهود نور

• حصة فاطمة = $1.3 \times 10 = 13$

الخلاصة :

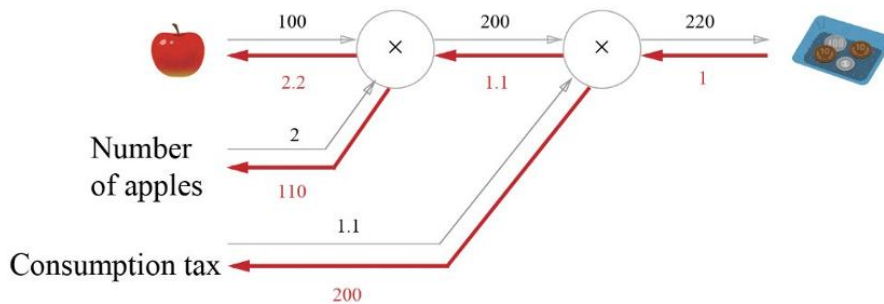
- عقدة الضرب ($\times$) هي مدير مشروع عادل
- عند توزيع الملاحظات، لا تعطي الجميع نفس الشيء، بل تنظر إلى مساهمة كل شخص وتعطيه ملاحظة بناءً على مجهود زميله.

لنرى المخططات :



لنعود لمثال شراء التفاح :

تخيل معي أننا محققون وقفنا أمام قضية شراء التفاح لقد حصلنا على النتيجة النهائية، وهي المبلغ الإجمالي المدفوع وهو 220 ين، لكن مهمتنا كمحققين هي أن نحدد قوة تأثير (مشتق) كل من المشتبه بهم الثلاثة على هذه النتيجة النهائية: لنرى المخطط الموافق لهذا المثال :



1. المشتبه به الأول: سعر التفاحة الواحدة

2. المشتبه به الثاني: عدد التفاح

3. المشتبه به الثالث: نسبة الضريبة

لحل هذه القضية، سنستخدم أداة التحقيق القوية: الانتشار الخلفي

سنبدأ من النتيجة النهائية (220 ين) ونتتبع قوة التأثير إلى الوراء، خطوة بخطوة، لنرى كيف وصلت إلى كل مشتبه به.

التحقيق يبدأ : الانتشار الخلفي خطوة بخطوة

الخطوة صفر: نقطة البداية (المبلغ النهائي) نبدأ تحقيقنا من النهاية.

نسأل: ما هي قوة تأثير المبلغ النهائي على نفسه؟

الإجابة المنطقية 1 هذه هي إشارة البداية التي سنطلقها

الخطوة الأولى: التحقيق مع عقدة الضريبة (1.1 ×) تصل إشارتنا (قيمة 1) إلى عقدة الضريبة, هذه العقدة هي عقدة

ضرب،

وقد تعلمنا قاعدتها : تبديل الأدوار

1. الإشارة القادمة من اليمين هي 1

2. العقدة تضرب هذه الإشارة في القيمة الأخرى التي كانت معها في عملية الضرب الأمامية، وهي المبلغ قبل الضريبة (200 ين).

3. الحساب: $200 = 1 \times 200$

4. النتيجة: هذه القيمة (200) هي قوة تأثير (مشتق) نسبة الضريبة

5. قوة تأثير المبلغ قبل الضريبة : $1.1 = 1 \times 1.1$ هذه الإشارة (1.1) هي التي سنمررها إلى اليسار

الخطوة الثانية:

التحقيق مع عقدة عدد التفاح (2 ×) تصل إشارة جديدة (قيمة 200) إلى عقدة عدد التفاح, هذه أيضاً عقدة ضرب، فنطبق نفس القاعدة.

1. الإشارة القادمة من اليمين هي 1.1

2. العقدة تضرب هذه الإشارة في القيمة الأخرى التي كانت معها، وهي سعر التفاحة الواحدة (100 ين)

3. قوة تأثير سعر التفاحة : $2.2 = 1.1 \times 2$

4. قوة تأثير عدد التفاح : $110 = 1.1 \times 100$

ماذا تعني هذه الأرقام؟

الآن لدينا قوة تأثير (مشتق) كل مشتبه به:

▪ قوة تأثير سعر التفاحة = 2.2

▪ قوة تأثير عدد التفاح = 110

▪ قوة تأثير نسبة الضريبة = 200

ماذا يعني هذا ؟

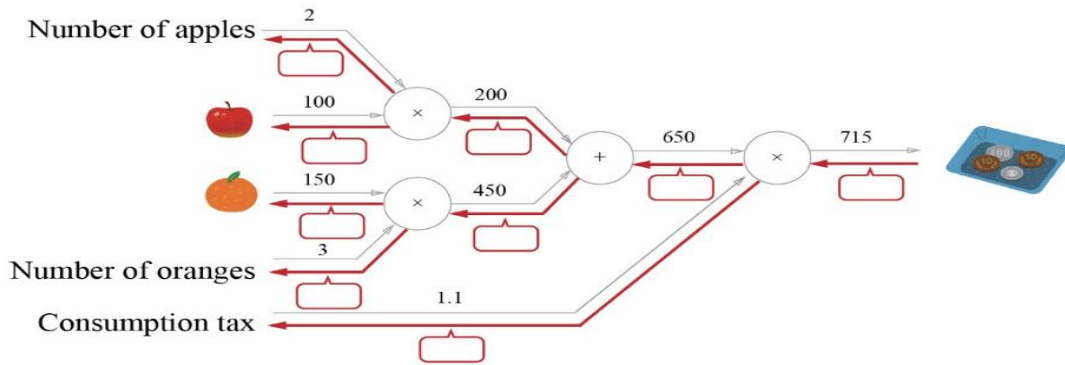
▪ سعر التفاحة : إذا زادت سعر التفاحة الواحدة بمقدار 1 ين، فإن المبلغ النهائي سيزداد بمقدار 2.2 ين

▪ عدد التفاح : إذا اشترت تفاحة واحدة إضافية، فإن المبلغ النهائي سيزداد بمقدار 110 ين

▪ نسبة الضريبة : إذا زادت نسبة الضريبة بمقدار 1.0 (أي 100٪)، فإن المبلغ النهائي سيزداد بمقدار 200 ين .

ولا ننسى أن الاسم باللون الاحمر هي التي تعبر عن الانتشار الخلفي .

وظيفة: مثال شراء التفاح والبرتقال يجب ملئ الفراغات التي تمثل الانتشار الخلفي بالمخطط



Implementing a Multiplication Layer بناء طبقات الشبكة العصبية

- تخيل أن المخططات الحسابية التي تعلمناها (مثل عقدة الضرب $\times$ والجمع $+$) هي مجرد مخططات على الورق
- الآن، حان الوقت لنحول هذه المخططات إلى قطع حقيقية يمكن فهمها والتعامل معها.
- هذه القطع الحقيقية تسمى طبقات (Layers) سنقوم ببناء طبقة للضرب (Multiplication Layer) وطبقة للجمع (Addition Layer).
- فكر في الأمر كأنك تبني قلعة من قطع الليغو. كل قطعة ليغو لها شكل محدد ووظيفة محددة. نحن الآن نصمم ونصنع قطعة الضرب وقطعة الجمع.

بناء قطعة الضرب (The Multiplication Layer):

لنبدأ بتصميم قطعة الليغو الخاصة بعملية الضرب

كل قطعة ليغو نصنعها من هذا القالب يجب أن يكون لديها وظيفتان أساسيتان:

- Forward المهمة الأمامية:** تأخذ مدخلات وتقوم بعملها.
 - Backward مهمة التغذية الراجعة:** تستقبل معلومات عن التعديلات المطلوبة وتحسب مساهمة كل مدخل.
- لنصنع MulLayer خطوة بخطوة:

```

3 class MulLayer:
4     def __init__(self):
5         self.x = None
6         self.y = None
7     def forward(self, x, y):
8         self.x = x
9         self.y = y
10        out = x * y
11        return out
12    def backward(self, dout):
13        dx = dout * self.y # Reverse x and y
14        dy = dout * self.x
15        return dx, dy

```

الجزء الأول: الإعداد والتحضير - (__init__)

- قبل أن يبدأ أي عامل عمله، يحتاج إلى أن يجهز مفكرة وقلماً. هذا ما تفعله هذه الدالة.
- تخيل أن طبقة الضرب هذه هي عامل في مصنع. وظيفته هي ضرب رقمين. قبل أن يبدأ وظيفته، يأخذ العامل مفكرة فارغة (__init__) ويجهز مكانين فارغين فيها: واحد ليكتب فيه الرقم الأول (self.x) والآخر للرقم الثاني (self.y)
- لماذا هذا مهم؟ لأن العامل سيحتاج إلى تذكر هذه الأرقام الأصلية لاحقاً عندما يأتيه طلب "تعديل" من مديره.

الجزء الثاني: تنفيذ المهمة (forward) العامل يؤدي عمله

- استلام المواد: يصل العامل وظيفته ويستلم رقمين x و y
- تدوين الملاحظات: أول شيء يفعله العامل هو أن يكتب الرقمين في مفكرته التي جهزها سابقاً
- (self.x = x, self.y = y)
- أداء العمل: يقوم بعمله الأساسي وهو ضرب الرقمين (out = x * y)
- تسليم المنتج: يسلم الناتج (out) إلى المحطة التالية في خط الإنتاج

الجزء الثالث: استقبال التغذية الراجعة (backward) العامل يستقبل ملاحظات المدير

- بعد فترة، يأتي مدير العامل من المحطة التالية (من اليمين) ويقول له: المنتج النهائي يحتاج إلى تعديل طفيف، قيمته dout
- إذا كان المنتج النهائي يحتاج إلى تعديل بقيمة dout، فكم من هذا التعديل يرجع إلى الرقم الأول (x) وكم يرجع إلى الرقم الثاني (y) ؟
- هنا يأتي دور المفكرة!

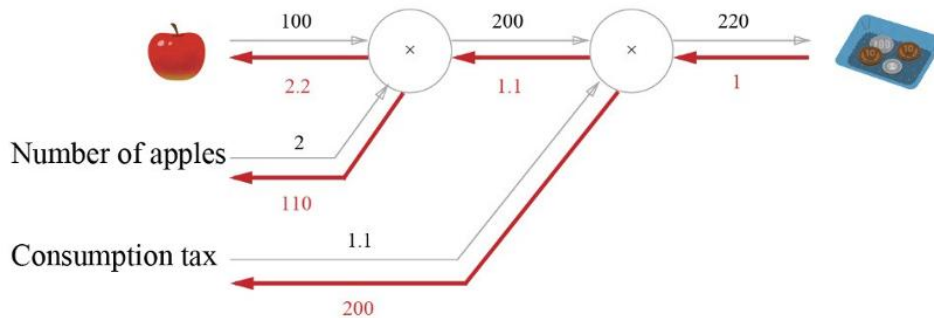
1. لحساب مساهمة الرقم الأول (dx) : يفتح العامل مفكرته، وينظر إلى الرقم الثاني (self.y) الذي كتبه هناك. ثم يضرب قيمة المدير (dout) في هذا الرقم (dout * self.y) هذا هو منطق تبديل الأدوار الذي تعلمناه سابقاً .
2. لحساب مساهمة الرقم الثاني (dy): ينظر إلى الرقم الأول (self.x) في مفكرته ويضرب قيمة المدير (dout) فيه (dout * self.x) .
3. إرسال الملاحظات: يرسل هذه الملاحظات (dx, dy) إلى المحطات السابقة (على اليسار) لتقوم هي الأخرى بتعديلاتها .

سؤال مهم :

- عند كتابة الكود البرمجي متى نستخدم للضرب * ومتى dot ؟
- نعتمد على العملية الامامية forward إذا كانت تستخدم العملية الامامية * نستخدمها في backward وإذا كانت تستخدم dot نستخدمها

مثال :

تخيل أننا انتقلنا من مرحلة رسم المخططات على الورق إلى مرحلة بناء آلة حقيقية مهمة هذه الآلة هي حساب سعر شراء التفاح



سنقوم ببنائها باستخدام قطع الليغو التي صممناها سابقاً (MulLayer)

```

17 apple = 100
18 apple_num = 2
19 tax = 1.1
20 # layer
21 mul_apple_layer = MulLayer()
22 mul_tax_layer = MulLayer()
23 # forward
24 apple_price = mul_apple_layer.forward(apple, apple_num)
25 print('Apple_price',apple_price)
26 price = mul_tax_layer.forward(apple_price, tax)
27 print('Price after tax : ',price)
28 # backward
29 dprice = 1
30 dapple_price, dtax = mul_tax_layer.backward(dprice)
31 dapple, dapple_num = mul_apple_layer.backward(dapple_price)
32 print('dapple : ',dapple )
33 print('dapple_num : ',dapple_num)
34 print('dtax : ',dtax)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Apple_price 200
Price after tax : 220.00000000000003
dapple : 2.2
dapple_num : 110.00000000000001
dtax : 200

```

1. الجزء الأول: تشغيل خط الإنتاج (Forward Propagation) :

هذا هو الجزء الذي تقوم فيه الآلة بالعمل الفعلي: حساب السعر النهائي.

الخطوة 1: إحضار الآلات وتجهيزها:

- `mul_apple_layer = MulLayer()`
- `mul_tax_layer() = MulLayer`

تخيل أن مصنعنا الصغير يحتاج إلى آتين للضرب:

- آلة سعر التفاح (`mul_apple_layer`) : مهمتها ضرب سعر التفاحة الواحدة في عددها
- آلة الضريبة (`mul_tax_layer`) : مهمتها ضرب المبلغ الإجمالي في نسبة الضريبة.

هنا، قمنا بطلب هاتين الآلتين وتثبيتهما في أماكنهما على خط الإنتاج. لا تزالان فارغتين وجاهزتين للعمل

الخطوة 2: تشغيل خط الإنتاج:

- # forward
- apple_price = mul_apple_layer.forward(apple, apple_num)
- price = mul_tax_layer.forward(apple_price, tax)

■ الآلة الأولى تعمل :

نأخذ المادة الخام الأولى وهي سعر التفاحة (apple = 100)

والمادة الثانية وهي عدد التفاح (apple_num = 2)

ونضعها في الآلة الأولى (mul_apple_layer) الآلة تعمل وتنتج منتج نصف مصنع وهو apple_price قيمته 200

تتذكر الآلة أنها استخدمت الرقمين 100 و 2

■ الآلة الثانية تعمل :

نأخذ المنتج النصف مصنع من الخطوة الأولى (apple_price = 200)

والمادة الخام الثالثة وهي نسبة الضريبة (tax = 1.1)

ونضعهما في الآلة الثانية (mul_tax_layer) هذه الآلة تعمل وتنتج المنتج النهائي وهو price قيمته 220
الآن لدينا المنتج النهائي: 220 ين .

2. الجزء الثاني: التحقيق في مصدر الخطأ (Backward Propagation) :

- الآن يأتي الجزء المثير. تخيل أن مدير الجودة جاء وقال: المنتج النهائي (220) به خطأ قيمته 1.
- نريد أن نعرف من المسؤول عن هذا الخطأ في المواد الخام .

الخطوة 1: بدء التحقيق :

- backward
- dprice = 1
- المدير يعطي إشارة خطأ بقيمة 1 للمنتج النهائي , هذه هي نقطة بداية التحقيق.

الخطوة 2: تتبع الخطأ إلى الخلف (بترتيب عكسي)

هذه هي أهم نقطة , التحقيق يسير عكس اتجاه خط الإنتاج تمامًا

- dapple_price, dtax = mul_tax_layer.backward(dprice)
- dapple, dapple_num = mul_apple_layer.backward(dapple_price)

1. التحقيق مع الآلة الثانية (الضريبة):

المدير يذهب أولاً إلى الآلة الأخيرة (mul_tax_layer) التي أنتجت المنتج النهائي.

يخبرها: إشارة الخطأ القادمة منك هي 1

الآلة تفتح ذاكرتها وتجد أنها استخدمت الرقمين 200 و 1.1.

فتقول: حسناً، إذا كان الخطأ النهائي 1، فإن مساهمة الضريبة (tax) هي $200 = 1 \times 200$ ، ومساهمة المبلغ

قبل الضريبة (apple_price) هي $1.1 = 1 \times 1.1$

تسجل مسؤولية الضريبة ($dtax = 200$) وتمرر إشارة الخطأ 1.1 إلى الورا، إلى الآلة التي سبقتها.

2. التحقيق مع الآلة الأولى (سعر التفاح):

المدير يأخذ إشارة الخطأ 1.1 ويذهب إلى الآلة الأولى (mul_apple_layer)

يخبرها: إشارة الخطأ القادمة منك هي 1.1

هذه الآلة تفتح ذاكرتها وتجد أنها استخدمت الرقمين 100 و 2

فتقول: حسناً، إذا كان الخطأ القادم مني 1.1، فإن مساهمة سعر التفاحة الواحدة (apple) هي $2.2 = 1.1 \times 2$ ،

ومساهمة عدد التفاح (apple_num) هي $110 = 1.1 \times 100$

تسجل مسؤوليتيهما ($dapple = 2.2$, $dapple_num = 110$)

الخلاصة النهائية للتحقيق

النتائج النهائية `print(dapple, dapple_num, dtax)`

ماذا تعني هذه الأرقام؟

- $dapple = 2.2$: سعر التفاحة مسؤول عن 2.2 درجة من الخطأ (إذا زاد سعر التفاحة بمقدار 1، يزيد المبلغ النهائي بمقدار 2.2).
- $dapple_num = 110$: عدد التفاح مسؤول عن 110 درجة من الخطأ (إذا زاد عدد التفاح بمقدار 1، يزيد المبلغ النهائي بمقدار 110).
- $dtax = 200$: نسبة الضريبة مسؤولة عن 200 درجة من الخطأ (إذا زادت الضريبة بمقدار 1، يزيد المبلغ النهائي بمقدار 200).

بناء طبقة الجمع (The Addition Layer)

```

33 class AddLayer:
34     def __init__(self):
35         pass
36     def forward(self, x, y):
37         out = x + y
38         return out
39     def backward(self, dout):
40         dx = dout * 1
41         dy = dout * 1
42         return dx, dy

```

- بعد أن صنعنا عامل الضرب الذي يحتاج إلى مفكرة، الآن سنصنع عامل الجمع البسيط والمباشر.

- هذا العامل هو الترجمة البرمجية لعقدة الجمع (+) في المخططات الحسابية.

الجزء الأول: الإعداد والتحضير (init) عامل لا يحتاج لأي شيء

- `def __init__(self)`
- تخيل أن طبقة الجمع هذه هي عامل في المصنع لكن مهمته بسيطة جداً لدرجة أنه لا يحتاج إلى أي إعداد
- عندما يأتي إلى العمل في الصباح، لا يأخذ معه مفكرة أو قلماً أو أي أدوات pass تعني لا تفعل شيئاً .

لماذا لا يحتاج لأي شيء؟

لأن مهمته في التحقيق الخلفي (المرحلة backward) لا تعتمد على تذكر الأرقام الأصلية التي جمعها، على عكس عامل الضرب الذي كان بحاجة إلى مفكرته. مهمته بسيطة جداً: فقط مرور الرسالة كما هي.

الجزء الثاني: تنفيذ المهمة (forward) العامل يجمع الصناديق

- `def forward(self, x, y):`
- `out = x + y`
- `return out`
- هذه هي المهمة الأساسية للعامل:

- استلام الصناديق: يصل العامل وريدته ويستلم صندوقين x و y
 - دمج المحتويات: يأخذ محتويات الصندوق الأول ويضيفها إلى محتويات الصندوق الثاني ($out = x + y$)
 - تسليم المنتج: يسلم الصندوق المدموج (out) إلى المحطة التالية في خط الإنتاج.
- مثلاً: إذا أعطيته صندوقاً به 10 تفاحات وآخر به 5 تفاحات، فإنه سيضع كل الـ 15 تفاحة في صندوق واحد كبير ويسلمه .

الجزء الثالث: استقبال التغذية الراجعة (backward) العامل الموزع للرسائل

- `def backward(self, dout):`
- `dx = dout * 1`
- `dy = dout * 1`
- `return dx, dy`
- تخيل أن مديره جاء من المحطة التالية (من اليمين) وقال له:
- هذه رسالة مهمة ($dout$) ، يجب أن توصلها إلى المصدرين اللذين أتيت منهما .

ماذا يفعل العامل؟

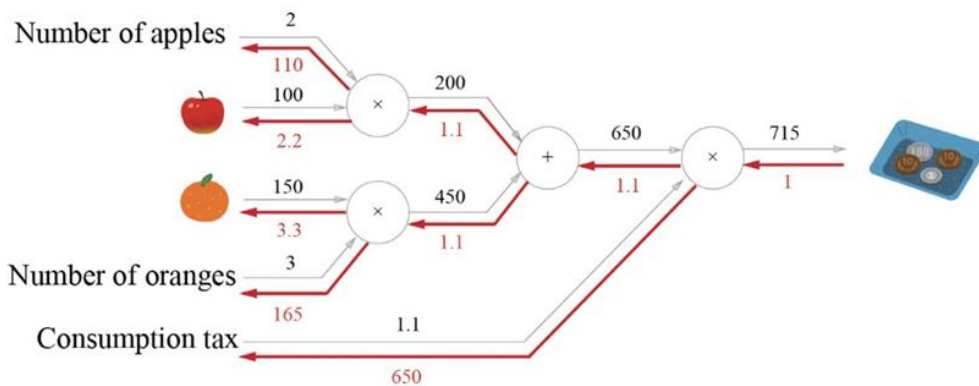
- يستلم الرسالة: يأخذ الرسالة ($dout$) من المدير.

- **يُصور نسخة:** لا يغير فيها شيئاً. يصور نسخة منها ($dout * 1$) ويسمى dx ويرسلها إلى المصدر الأول (x)
- **يُصور نسخة أخرى:** يصور نسخة أخرى مطابقة تماماً ($dout * 1$) ويسمى dy ويرسلها إلى المصدر الثاني (y)

الخلاصة:

- عامل الجمع هو مجرد موزع رسائل أي رسالة تأتيه من اليمين، يقوم بتوزيعها كما هي إلى اليسار دون أي تغيير.
- هذا يطابق تماماً ما تعلمناه سابقاً: مشتق عملية الجمع هي 1. لذلك، هو يضرب الرسالة القادمة في 1 (وهذا لا يغيرها) ويرسلها .

لقد تعلمنا كيف نضع آلة ضرب وآلة جمع. الآن، سنستخدم هذه الآلات البسيطة لبناء مصنع أكبر وأكثر تعقيداً: مصنع يحسب سعر شراء التفاح والبرتقال معاً .



```

44 apple = 100
45 apple_num = 2
46 orange = 150
47 orange_num = 3
48 tax = 1.1
49 # layer
50 mul_apple_layer = MulLayer()
51 mul_orange_layer = MulLayer()
52 add_apple_orange_layer = AddLayer()
53 mul_tax_layer = MulLayer()

```

```

55 # forward
56 apple_price = mul_apple_layer.forward(apple, apple_num)
57 print("Apple_price : ",apple_price)
58 orange_price = mul_orange_layer.forward(orange, orange_num)
59 print('Orange_price : ',orange_price)
60 all_price = add_apple_orange_layer.forward(apple_price, orange_price)
61 print('All_price : ' , all_price)
62 price = mul_tax_layer.forward(all_price, tax)
63 print('Price after tax : ',price)
64 # backward
65 dprice = 1
66 dall_price, dtax = mul_tax_layer.backward(dprice)
67 dapple_price, dorange_price = add_apple_orange_layer.backward(dall_price)
68 dorange, dorange_num =mul_orange_layer.backward(dorange_price)
69 dapple, dapple_num =mul_apple_layer.backward(dapple_price)
70 print('Dapple_num : ',dapple_num)
71 print('Dapple : ',dapple,)
72 print('Dorange : ', dorange)
73 print('Dorange_num :', dorange_num)
74 print('Dtax : ', dtax)

```

```

Apple_price : 200
Orange_price : 450
All_price : 650
Apple_price : 200
Orange_price : 450
Apple_price : 200
Apple_price : 200
Orange_price : 450
All_price : 650
Price after tax : 715.00000000000001
Dapple_num : 110.00000000000001
Dapple : 2.2
Dorange : 3.3000000000000003
Dorange_num : 165.0
Dtax : 650

```

1. الجزء الأول: بناء وتشغيل المصنع المعقد (الانتشار الأمامي forward)

المخطط يوضح عملية شراء التفاح والبرتقال هو المخطط الهندسي لمصنعنا الجديد. لبنى هذا المصنع خطوة بخطوة.

الخطوة 1: إحضار الآلات وتثبيتها

- mul_apple_layer = MulLayer()
- mul_orange_layer = MulLayer()
- add_apple_orange_layer = AddLayer()
- mul_tax_layer()= MulLayer

لتنفيذ هذا المخطط المعقد، نحتاج إلى 4 آلات

1. آلة ضرب التفاح (mul_apple_layer)

2. آلة ضرب البرتقال (mul_orange_layer)

3. آلة جمع الفواكه (add_apple_orange_layer)

4. آلة ضرب الضريبة (mul_tax_layer)

قمنا بطلب هذه الآلات الأربع وتثبيتها في أماكنها الصحيحة على خط الإنتاج الجديد.

الخطوة 2: تشغيل خط الإنتاج (بترتيب منطقي)

forward

- apple_price = mul_apple_layer.forward(apple, apple_num)
- orange_price = mul_orange_layer.forward(orange, orange_num)
- all_price = add_apple_orange_layer.forward(apple_price, orange_price)
- price = mul_tax_layer.forward(all_price, tax)

الآن نشغل المصنع. لاحظ أن كل آلة تعتمد على مخرجات الآلة التي قبلها.

- **الخطوتان (1) و (2):** تبدأ آلتا التفاح والبرتقال بالعمل في نفس الوقت (بشكل متواز)
 - آلة التفاح تستلم سعر التفاحة (100) وعددها (2) وتنتج سعر التفاح الإجمالي (200), (تتذكر القيمتين 100 و (2).
 - آلة البرتقال تستلم سعر البرتقالة (150) وعددها (3) وتنتج سعر البرتقال الإجمالي (450), (تتذكر القيمتين 150 و (3).
 - **الخطوة (3):** تأخذ آلة الجمع المنتجين شبه المصنعين (200 و 450) وتدمجها معًا لتنتج سعر الفواكه قبل الضريبة (650)
 - **الخطوة (4):** أخيرًا، تأخذ آلة الضريبة المنتج (650) وتضربه في نسبة الضريبة (1.1) لتنتج المنتج النهائي والسعر الإجمالي (715)
- لقد عمل المصنع بكفاءة وأنتج لنا النتيجة النهائية.

الجزء الثاني: التحقق في جودة المنتج النهائي (الانتشار الخلفي backward)

- الآن يأتي دور التحقق
- مدير الجودة يأتي ويقول: المنتج النهائي (715) له خطأ مقداره 1
- (dprice = 1) اكتشفوا مصدر الخطأ
- **القاعدة الذهبية:** التحقق يسير بترتيب عكسي تمامًا لتشغيل المصنع .
- # backward
- dprice = 1
- dall_price, dtax = mul_tax_layer.backward(dprice)
- dapple_price, dorange_price = add_apple_orange_layer.backward(dall_price)

- `dorange, dorange_num = mul_orange_layer.backward(dorange_price)`
- `dapple, dapple_num = mul_apple_layer.backward(dapple_price)`

التحقيق مع الآلة الأخيرة (4) :

1. يذهب المحقق إلى آلة الضريبة (الآلة رقم 4) يخبرها:
2. قيمة الخطأ القادمة منك هي 1 الآلة تفتح ذاكرتها (تتذكر أنها استخدمت 650 و 1.1) وتحسب المسؤولية
3. مسؤولية الضريبة هي $650 = 1 \times 650$ ، ومسؤولية المبلغ قبل الضريبة هي $1.1 = 1 \times 1.1$
4. تمرر قيمة 1.1 إلى الآلة التي سبقتها

التحقيق مع آلة الجمع (3) :

1. يأخذ المحقق قيمة التأثير 1.1 ويذهب إلى آلة الجمع
2. هذه الآلة البسيطة لا تغير شيئاً، بل تمرر قيمة التأثير كما هي
3. مسؤولية سعر التفاح الإجمالي هي 1.1، ومسؤولية سعر البرتقال الإجمالي هي 1.1 .

التحقيق مع آلة البرتقال (2) :

1. يأخذ المحقق قيمة التأثير 1.1 (المسؤولية عن سعر البرتقال)
2. ويذهب إلى آلة ضرب البرتقال تفتح الآلة ذاكرتها (تتذكر أنها استخدمت 150 و 3)
3. وتحسب: مسؤولية سعر البرتقالة الواحدة هي $3.3 = 1.1 \times 3$ ، ومسؤولية عدد البرتقال هي $165 = 1.1 \times 150$

التحقيق مع آلة التفاح (1) أخيراً :

1. يأخذ المحقق قيمة التأثير 1.1 (المسؤولية عن سعر التفاح)
2. ويذهب إلى آلة ضرب التفاح. تفتح الآلة ذاكرتها (تتذكر أنها استخدمت 100 و 2)
3. وتحسب: مسؤولية سعر التفاحة الواحدة هي $2.2 = 1.1 \times 2$ ، ومسؤولية عدد التفاح هي $110 = 1.1 \times 100$

الخلاصة:

قوة التصميم المعياري (Modular Design)
النتائج النهائية (650 3.3 2.2 110) تطابق تماماً ما حسبناه يدوياً في المخططات.
ماذا تعلمنا؟

- لقد بنينا نظاماً معقداً عن طريق ربط آلات بسيطة ومستقلة معاً.
 - كل آلة تعرف وظيفتها فقط في forward و backward هذا التصميم قوي جداً لأنه:
 - منظم: كل جزء له مسؤوليته الواضحة.
 - مرن: يمكننا إضافة أو إزالة آلات بسهولة لتغيير المصنع.
 - قابل للتوسع: يمكننا بناء مصانع ضخمة جداً (شبكات عصبية عميقة) بنفس المبدأ.
- هذا هو جوهر بناء الشبكات العصبية: بناء طبقات بسيطة وربطها معاً لحل مشاكل معقدة.

Implementing the Activation Function Layer

- الآن، بعد أن أتقننا الآلات الأساسية (الجمع والضرب)
- حان الوقت لبناء آلات أكثر تخصصاً وذكاءً، يتم بناء الشبكة العصبية كوحدات تسمى "طبقات" (Layers) ، حيث تحتوي كل طبقة على طريقة للانتشار الأمامي (forward()) وطريقة للانتشار الخلفي (backward())
- مثل طبقات ReLU و Sigmoid التي تعتبر من أهم مكونات الشبكات العصبية الحقيقية .

ReLU Layer

تخيل أن طبقة ReLU هي حارس أمن ذكي جداً يقف عند باب مبنى مهم (مثل شبكتنا العصبية). مهمته هي فلترة الأشخاص (الإشارات) التي تريد الدخول.

قاعدة الحارس الأساسية:

- سأسمح فقط للأشخاص الذين لديهم طاقة إيجابية (قيمتهم أكبر من صفر) بالدخول.
- أي شخص لديه طاقة صفرية أو سلبية، سأمنعه من الدخول وقيمتهم ستصبح صفر
- المعادلة :

$$y = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

- نشتق y بالنسبة ل x فنحصل على :

$$\frac{\partial y}{\partial x} = \begin{cases} 1 & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

1. الجزء الأول: عمل الحارس في اليوم العادي (الانتشار الأمامي forward)

هذا هو عمل الحارس عند مرور الأشخاص (البيانات) من اليسار إلى اليمين.

الحالة الأولى: شخص ذو طاقة إيجابية(+5)

- يصل شخص قيمته +5 إلى الحارس
- يفحصه الحارس ويجد أن طاقته إيجابية
- يسمح له بالدخول دون أي تغيير. القيمة الخارجة هي +5

الحالة الثانية: شخص ذو طاقة سالبة(-3)

- يصل شخص قيمته -3 إلى الحارس
- يفحصه الحارس ويجد أن طاقته سالبة
- يمنعه من الدخول. القيمة الخارجة هي 0

الحالة الثالثة: شخص ذو طاقة صفرية(0)

- يصل شخص قيمته 0 إلى الحارس
- يفحصه الحارس ويجد أن طاقته ليست إيجابية
- يمنعه من الدخول. القيمة الخارجة هي 0
- هذه هي وظيفة forward ببساطة :
- القيمة الخارجة $max =$ (القيمة الداخلة, 0)
- تأخذ القيمة الأكبر بين الصفر والقيمة الأصلية.

2. الجزء الثاني: تقييم أداء الحارس (الانتشار الخلفي backward)

- الآن، تخيل أن المدير (عملية التعلم) جاء من داخل المبنى (من اليمين) وقال للحارس:
- لقد حدث خطأ ما في الداخل نريد أن نرسل إشارة تصحيح إلى الخارج لمعرفة المسؤولين.
- مهمتك هي تمرير هذه الإشارة إلى الورا
- هنا، يتصرف الحارس بذكاء بناءً على ذاكرته للشخص الذي فحصه.
- **الحالة الأولى: الشخص الذي دخل (كانت قيمته موجبة)**
- يتذكر الحارس أنه سمح لهذا الشخص بالدخول لأنه كان إيجابياً
- قاعدة الحارس هنا:
- لقد تركت هذا الشخص يمر دون تغيير، لذلك فهو مسؤول كلياً عن أي خطأ يأتي من الداخل.
- سأمرر إشارة التصحيح إليه كما هي بالضبط
- هذا هو السبب في أن مشتقة ReLU عندما تكون $x > 0$ هي 1 الإشارة تمرر كاملة ($dout * 1$)

الحالة الثانية: الشخص الذي تم منعه (كانت قيمته صفرية أو سالبة)

- يتذكر الحارس أنه منع هذا الشخص من الدخول لأنه لم يكن إيجابياً
- قاعدة الحارس هنا:
- هذا الشخص لم يدخل المبنى إطلاقاً ومساهمته في الداخل كانت صفر
- بالتالي، لا يمكن أن يكون مسؤولاً عن أي خطأ حدث في الداخل
- إشارة التصحيح ستتوقف عندي ولن تصل إليه
- هذا هو السبب في أن مشتقة ReLU عندما تكون $x \leq 0$ هي 0 الإشارة تموت ($dout * 0$)

لماذا هذا الحارس شائع جداً؟

- طبقة ReLU هي من أشهر طبقات التنشيط في الشبكات العصبية لسببين رئيسيين:
- البساطة والسرعة: قاعدته سهلة جداً للحساب (إما تمرر القيمة أو تجعلها صفر)، مما يجعله سريعاً وفعالاً جداً.
- الكفاءة في التعلم: على عكس الحراس الآخرين الذين قد يضعفون إشارة التصحيح، هذا الحارس إما أن يمرر الإشارة بقوتها الكاملة أو يوقفها تماماً. هذا يحافظ على قوة إشارة التعلم قوية أثناء انتقالها للخلف عبر الشبكة، مما يساعد على تعلم أسرع وأفضل.

تعميم:

- إذا كان المدخل في الانتشار الأمامي x أكبر من 0، فإن الانتشار العكسي يمرر القيمة من اليمين إلى اليسار دون تغييرها .

- في الوقت نفسه: إذا كانت x تساوي 0 أو أقل في الانتشار الأمامي، فإن الإشارة تتوقف هناك في الانتشار العكسي أي 0.

أي :

- إذا كان $x > 0$ في الانتشار الأمامي: $\frac{\partial y}{\partial x} = 1$ (تمرير الإشارة الخلفية كما هي)
 - إذا كان $x \leq 0$ في الانتشار الأمامي: $\frac{\partial y}{\partial x} = 0$ (تمرير الإشارة الخلفية بصفر، أي إيقاف الانتشار)
- كما هو موضح في المخطط التالي



بعد ان فهمنا فكرة عمل الحارس (ReLU) لنقوم بتنفيذ الفكرة :

```

81 class Relu:
82     def __init__(self):
83         self.mask = None
84
85     def forward(self, x):
86         self.mask = (x <= 0)
87         out = x.copy()
88         out[self.mask] = 0
89         return out
90     def backward(self, dout):
91         dout[self.mask] = 0
92         dx = dout
93         return dx

```

1. الجزء الأول: تجهيز الحارس (__init__) الحارس يجهز قائمته mask

- def __init__(self) :
- هذه هي دالة بداية الدوام الخاصة بالحارس
- أول ما يفعله عندما يبدأ عمله :
- self.mask = None :
- يجهز دفتر ملاحظات فارغ
- بالبداية يكون فارغ
- الغرض منه سيستخدمه الحارس لاحقاً ليسجل فيه الأشخاص الذين منعهم من الدخول .

2. الجزء الثاني: الحارس يؤدي عمله (forward) يوم الحارس في العمل

```

def forward(self, x):
    self.mask = (x <= 0)
    out = x.copy()
    out[self.mask] = 0
    return out

```

1. تعريف دالة المرور الأمامي حيث x هي قائمة الأشخاص القادمة إلى الحارس
2. هذا هو سطر اتخاذ القرار وتسجيله :

- الحارس ينظر إلى كل شخص في القائمة x ويتخذ قراراً سريعاً:
- هل طاقة هذا الشخص (قيمه) أقل من أو تساوي صفر؟
- النتيجة: ينشئ قائمة جديدة من True و False ويخزنها في دفتر ملاحظاته (self.mask)
- True : يعني سأمنعه
- False : يعني سأسمح له بالمرور
- الحارس يأخذ صورة من قائمة الأشخاص الأصلية x ويسمونها out هذا مهم جداً لعدم التغيير في القائمة الأصلية .
- هذا هو سطر تنفيذ قرار المنع.
- الحارس ينظر إلى دفتر ملاحظاته (self.mask)
- لكل شخص مسجل فيه ب True (أي أنه كان يجب منعه)، يذهب إلى القائمة الجديدة out ويغير قيمته إلى صفر.
- بعبارة أخرى: كل الأشخاص الذين قررت منعهم، قيمتهم النهائية الآن صفر .
- الحارس يسلم القائمة النهائية out (التي تحتوي على الأشخاص الذين مروا بقيمتهم الأصلية، والأشخاص الذين تم منعهم وأصبحت قيمتهم صفر) إلى المحطة التالية في المبنى.

3. الجزء الثالث: الحارس يقدم تقريراً (الانتشار الخلفي backward)

```

def backward(self, dout):
    dout[self.mask] = 0
    dx = dout
    return dx

```

1. تعريف دالة المرور الخلفي حيث $dout$ هي إشارة خطأ قادمة من داخل المبنى
2. هنا يكمن ذكاء الحارس يستخدم دفتر ملاحظاته مرة أخرى
- الحارس ينظر إلى دفتر ملاحظاته (self.mask) الذي جهزه في مرحلة forward
- لكل شخص مسجل فيه ب True (أي أنه لم يدخل المبنى إطلاقاً)، يقول: هذا الشخص لا يمكن أن يكون مسؤولاً عن أي خطأ حدث داخل المبنى

- لذلك، يأخذ إشارة الخطأ (dout) ويجعل قيمة هذا الشخص فيها صفر. إشارة الخطأ لا تمر عبره.
- بعد أن حجب إشارة الخطأ عن الأشخاص غير المسؤولين، يمرر الحارس بقية إشارة الخطأ (الخاصة بالأشخاص الذين سُمح لهم بالدخول) إلى الخارج.
- الحارس يسلم تقرير المسؤولية النهائي (dx) إلى المرحلة السابقة (على اليسار) لكي تعرف هي الأخرى من المسؤول عن الخطأ

والآن

تخيل أن طبقة ReLU ليست حارساً، بل هي مفتاح كهربائي ذكي جداً موجود في دائرة معقدة (شبكتنا العصبية). هذا المفتاح له حالتان فقط ON مفتوح OFF مغلق

1. الجزء الأول mask : ذاكرة المفتاح (The Switch's Memory)

قبل أن يتمكن المفتاح من العمل، يحتاج إلى أن يعرف حالة التيار الكهربائي في كل سلك (كل قيمة مدخلة) هذا هو بالضبط دور المتغير mask

ما هو mask؟ هو لوحة ذاكرة (array) يسجل فيها المفتاح حالة كل سلك (كل قيمة في مصفوفة المدخلات x).

كيف تملأ هذه الذاكرة؟ عندما يصل إشارة التيار (x)، يقوم المفتاح بفحص كل سلك على حدة:

- إذا كانت قيمة السلك صفر أو سالبة ($x \leq 0$)، يكتب في لوحة الذاكرة mask قيمة True
- True تعني أن المفتاح في هذا السلك سيكون في حالة OFF مغلق
- إذا كانت قيمة السلك موجبة، يكتب في لوحة الذاكرة mask قيمة False
- False تعني أن المفتاح في هذا السلك سيكون في حالة ON مفتوح

مثال :

```
94 import numpy as np
95 x = np.array( [[1.0, -0.5], [-2.0, 3.0]] )
96 print('X : ',x)
97 mask = (x <= 0)
98 print('Mask : ',mask)
```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
X : [[ 1. -0.5]
      [-2.  3.]]
Mask : [[False  True]
        [ True False]]
```

2. الجزء الثاني forward : تشغيل المفتاح (Turning the Switch On/Off)

الآن المفتاح جاهز للعمل بناءً على ذاكرته.

- كيف يعمل؟ المفتاح ينظر إلى لوحة الذاكرة (mask) التي جهزها في كل سلك مكتوب فيه False المفتاح ON ، يسمح للتيار بالمرور دون أي تغيير.

في كل سلك مكتوب فيه True (المفتاح OFF)، يمنع مرور التيار تماماً بقيمة الخرج من هذا السلك تصبح صفر.

■ هذا هو بالضبط ما يفعله الكود $out[self.mask] = 0$:

في كل موضع كانت فيه الذاكرة mask قيمتها True المفتاح كان OFF، اجعل قيمة الخرج out تساوي صفر

3. الجزء الثالث: backward تتبع التيار للخلف (Tracing the Current Backwards)

تخيل أن مشكلة حدثت في نهاية الدائرة (من اليمين) ووصلت إشارة خطأ (dout) نريد الآن أن نرسل هذا التيار الخاطئ للخلف لمعرفة مصدره.

- كيف يتصرف المفتاح؟ المفتاح ينظر مرة أخرى إلى لوحة الذاكرة (mask) التي جهزها في مرحلة forward
- 1. في كل سلك مكتوب فيه True المفتاح كان OFF
 - يقول المفتاح: كنت مغلقاً، لذلك لا يمكن أن يكون التيار الذي مر مني هو سبب الخطأ الذي حدث بعدي. المسؤولية ليست عليّ
 - لذلك، يمنع إشارة الخطأ من المرور ($dout[self.mask] = 0$)
- في كل سلك مكتوب فيه False المفتاح كان ON
 - يقول المفتاح: كنت مفتوحاً، لذلك أنا مسؤول عن مرور التيار
 - سأسمح لـ إشارة الخطأ بالمرور عبري كما هي ($dx = dout$)

الخلاصة النهائية:

- طبقة ReLU هي مفتاح ذكي
- يتذكر حالته من مرحلة المرور الأمامي أي الأسلاك كانت ON أو OFF
- ويستخدم هذه الذاكرة بالضبط للتحكم في تدفق إشارة الخطأ في مرحلة المرور الخلفي.
- هذا السلوك البسيط (إما تمرير كامل أو منع تام) هو ما يجعله فعالاً جداً وقوياً في الشبكات العصبية.

Sigmoid Layer

تخيل أن طبقة Sigmoid هي مترجم احتمالات ذكي مهمته ليست بسيطة مثل السماح أو المنع، بل هي أكثر دقة.

قاعدة المترجم الأساسية:

أعطني أي درجة خام (سواء كانت كبيرة جداً أو صغيرة جداً أو حتى سالبة)، وسأترجمها لك إلى احتمال يتراوح دائماً بين 0 و 1

مثال:

- إذا أعطيته درجة عالية جداً (مثل 10)، سيترجمها إلى احتمال قريب جداً من 1 (مثل 0.999).
- إذا أعطيته درجة منخفضة جداً (مثل -10)، سيترجمها إلى احتمال قريب جداً من 0 (مثل 0.0001).
- إذا أعطيته درجة صفر، سيترجمها إلى احتمال 0.5 (يعني "غير متأكد").

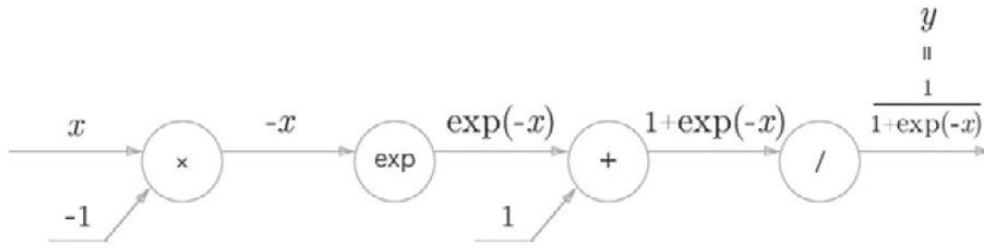
1. الجزء الأول: الوصفة السرية للمترجم

المعادلة الرياضية :

$$y = \frac{1}{1 + \exp(-x)}$$

النتيجة النهائية هي الاحتمال المترجم (y)

2. الجزء الثاني: بناء آلة الترجمة (المرور الامامي)



يمثل المخطط التالي كيفية تكوين المعادلة الرياضية :

الهدف هو حساب قيمة y عند إعطائها قيمة x يتبع المخطط الخطوات التالية:

1. عقدة الضرب (Multiplication):

- المدخل هو x
- تتم عملية ضرب x في -1
- الناتج $a = -x$ أو $a = -1 * x$

2. عقدة الأس (Exponentiation):

- المدخل هو الناتج من الخطوة السابقة a
- تتم عملية حساب الأس الطبيعي (e) للقيمة a
- الناتج $b = \exp(a) = e^{-x}$

3. عقدة الجمع (Addition):

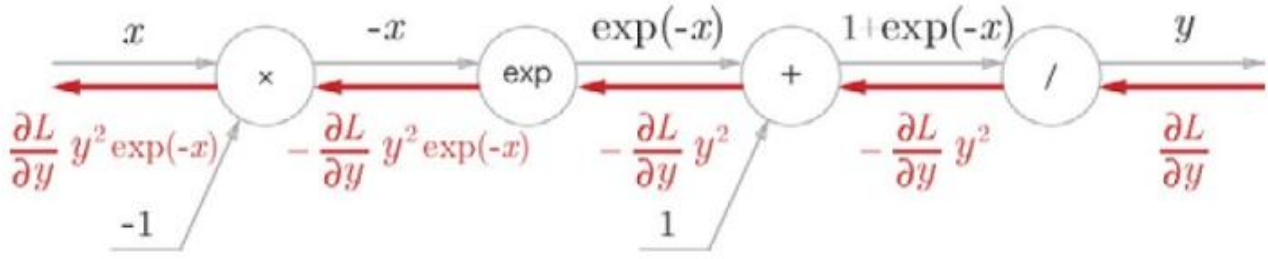
- المدخل هو الناتج b من الخطوة السابقة، ويضاف إليه الرقم 1
- الناتج $c = 1 + b = 1 + e^{-x}$

4. عقدة القسمة (Division):

- المدخل هو الناتج c من الخطوة السابقة
- تتم عملية قسمة الرقم 1 على c
- الناتج النهائي $y = 1 / c = 1 / (1 + e^{-x})$

هنا ظهر لدينا عقدتين جديدتين هما / و exp .

3. الجزء الثالث: كيف يتعلم المترجم من أخطائه (الانتشار الخلفي)



- هذا هو الجزء الأهم
 - الهدف هنا هو معرفة مدى تأثير تغيير المدخل x على الخطأ النهائي في الشبكة نستخدم لذلك قاعدة السلسلة (Chain Rule)
 - حيث نحسب المشتق عند كل عقدة ونضربها معاً.
- لنفترض أننا نعرف المشتق القادم من الطبقة التالية، وهي $\partial L / \partial y$ تأثير y على الخطأ Δ سنبدأ من النهاية ونرجع للخلف:

1. عقدة القسمة ($y = \frac{1}{c}$):

▪ مشتق y بالنسبة لـ c هو $\frac{\partial y}{\partial c} = -1/c^2$

▪ ملاحظة: بما أن $y = 1/c$:

▪ فإن $y^2 = 1/c^2$ لذلك، يمكننا كتابة المشتق كـ $\frac{\partial y}{\partial c} = -y^2$ هذا يسهل الحسابات لاحقاً لأننا نعرف قيمة y بالفعل من الانتشار الأمامي.

▪ التدرج الذي يخرج من العقدة:

▪ $\partial L / \partial c = \partial L / \partial y * \partial y / \partial c = \partial L / \partial y * (-1/c^2)$ هذا التدرج ينتقل الآن إلى العقدة السابقة (عقدة الجمع).

2. عقدة الجمع ($c = 1 + b$):

▪ مشتق c بالنسبة لـ b هو $\frac{\partial c}{\partial b} = 1$ مشتق الثابت 1 هي 0، ومشتق b هي 1

▪ التدرج الذي يخرج من العقدة:

▪ ببساطة، التدرج يمر عبر هذه العقدة دون تغيير. $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} * \frac{\partial c}{\partial b} = \frac{\partial L}{\partial c} * 1 = \frac{\partial L}{\partial c}$

3. عقدة الأس ($b = e^a$):

▪ مشتق b بالنسبة لـ a هي $\frac{\partial b}{\partial a} = e^a$ ومما سبق، نعلم أن $b = e^a$.

▪ التدرج الذي يخرج من العقدة:

▪ $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial b} * \frac{\partial b}{\partial a} = \frac{\partial L}{\partial b} * b$

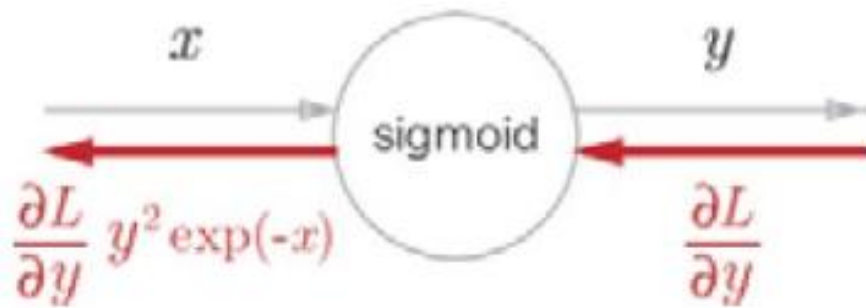
4. عقدة الضرب ($a = -1 * x$):

▪ مشتق a بالنسبة لـ x هو $\frac{\partial a}{\partial x} = -1$

▪ التدرج النهائي الذي نبحث عنه:

▪ $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} * \frac{\partial a}{\partial x} = \frac{\partial L}{\partial a} * (-1)$

يمكننا رسم المخطط السابق بعقدة واحدة اسمها Sigmoid كالتالي:



التبسيط النهائي والصيغة الموحدة

الآن، لنجمع كل هذه الخطوات معاً باستخدام قاعدة السلسلة :

$$\begin{aligned} \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial y} * \left(\frac{\partial y}{\partial c}\right) * \left(\frac{\partial c}{\partial b}\right) * \left(\frac{\partial b}{\partial a}\right) * \left(\frac{\partial a}{\partial x}\right) \\ \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial y} * (-1/c^2) * (1) * (b) * (-1) \\ \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial y} * (b / c^2) \end{aligned}$$

هذه هي الصيغة الكاملة. لكن يمكننا تبسيطها أكثر باستخدام قيم y و c و b التي عرفناها من الانتشار الأمامي:

$$\begin{aligned} \text{نعرف أن } y &= 1/c \\ \text{نعلم أن } b &= c - 1 \leftarrow c = 1 + b \text{ إذاً } b = c - 1 \\ \frac{b}{c} &= \frac{c-1}{c} = \left(\frac{c}{c}\right) - \left(\frac{1}{c}\right) = 1 - \left(\frac{1}{c}\right) = 1 - y \\ \text{إذاً، يمكننا إعادة كتابة } b/c^2 &\text{ كالتالي :} \\ b / c^2 &= (b/c) * (1/c) = (1 - y) * y \end{aligned}$$

لذلك، تصبح المعادلة النهائية للتدرج :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} * y * (1 - y)$$

الآن لنرى الطبقة :

```

101 class Sigmoid:
102     def __init__(self):
103         self.out = None
104     def forward(self, x):
105         out = 1 / (1 + np.exp(-x))
106         self.out = out
107         return out
108     def backward(self, dout):
109         dx = dout * (1.0 - self.out) * self.out
110         return dx

```

أولاً : الإعداد الأولي للمهمة (__init__)

1. def __init__(self):

- هذه هي الدالة التي تُستدعى تلقائياً عند إنشاء كائن جديد من السيجمويد. فكر فيها كـ إعدادات المصنع
- 2. self.out = None
- هنا self تعني الكائن نفسه نحن نعطي كل كائن سيجمويد ننشئه ذاكرة صغيرة اسمها out في البداية، هذه الذاكرة فارغة (None).

لماذا نحتاج لهذه الذاكرة؟

كما رأينا في المخطط، عملية الانتشار الخلفي (backward) تحتاج إلى قيمة المخرجات y من عملية الانتشار الأمامي (forward) لحساب المشتق هذه الذاكرة self.out هي المكان الذي سنخزن فيه قيمة y مؤقتاً لاستخدامها لاحقاً.

ثانياً: مرحلة الحساب (forward)

1. def forward(self, x):

2. out = 1 / (1 + np.exp(-x))

3. self.out = out

4. return out

حساب المخرجات بناءً على المدخلات

- 3. بعد حساب القيمة النهائية (out) ، لا نرجعها فقط بل نحفظ نسخة منها في ذاكرة الكائن (self.out) الآن الكائن تذكر قيمة مخرجاته
- 4. نرجع القيمة المحسوبة للطبقة التالية في الشبكة العصبية لتكمل عملها .

ثالثاً: مرحلة التعلم (backward) :

1. def backward(self, dout):

2. dx = dout * (1.0 - self.out) * self.out

3. return dx

الانتشار الخلفي، أي حساب التدرج (المشتق) لنشر الخطأ وتحديث الأوزان

- 1. الدالة تأخذ مدخلاً اسمه dout هذا هو التدرج القادم من الطبقة التالية، وهو نفسه $\partial L / \partial y$ الذي تحدثنا عنه تأثير مخرجاتنا y على الخطأ الكلي L
- 2. هذا هو قلب العملية! لنقارنه بالمعادلة التي توصلنا إليها من المخطط:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} * y * (1 - y)$$

• dout هو $\partial L / \partial y$

• self.out هو القيمة y التي حفظناها في مرحلة forward

• 1 - self.out هو (1 - y)

- 3. نرجع التدرج المحسوب $(\frac{\partial L}{\partial x})$ إلى الطبقة السابقة في الشبكة لتكمل هي الأخرى عملية التعلم

مثال لتوضيح الفكرة

تخيل أن class Sigmoid هو موظف في شركة

- `__init__` : عند تعيين الموظف، تعطيه دفتر ملاحظات صغير (`self.out = None`)
- `forward(x)` تعطيه مهمة (`x`) يقوم بالحسابات، يكتب الإجابة النهائية في دفتر ملاحظاته (`self.out = out`) ، ثم يسلمك الإجابة (`return out`)
- `backward(dout)` لاحقاً تعطيه ملاحظات وتصحيحاً على عمله (`dout`) لكي يعرف كيف يتحسن في المرة القادمة (`dx`) ، لا ينظر فقط للملاحظات، بل يفتح دفتر ملاحظاته وينظر للإجابة التي كتبها سابقاً (`self.out`) ليقارن ويحسب الخطأ بدق .

Implementing the Affine and Softmax Layers

Affine Layer

- تخيل أنك أمام لوحة تحكم معقدة لآلة ضخمة جداً (مثل ذراع روبوت في مصنع).
- هذه اللوحة ليس فيها زر واحد، بل بها عدة مؤشرات وإعدادات.
- هذه اللوحة هي بالضبط ما تمثله طبقة Affine في الشبكات العصبية

```
103 X = np.random.rand(2) # Input values
104 W = np.random.rand(2,3) # Weights
105 B = np.random.rand(3) # Biases
106 print('Shape of x is : ', X.shape)
107 print('Shape of w is : ', W.shape)
108 print('Shape of b is : ', B.shape)
109 Y = np.dot(X, W) + B
```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Shape of x is : (2,)
Shape of w is : (2, 3)
Shape of b is : (3,)
```

الشرح :

الجزء الأول: مكونات لوحة التحكم

لنفهم كيف تعمل هذه اللوحة، يجب أن نعرف مكوناتها الثلاثة:

Inputs المدخلات X

هذه هي مؤشرات التحكم التي يمكنك تحريكها.

المعنى الحقيقي: هي البيانات الأولية التي تدخل إلى الشبكة العصبية (مثل بكسلات من صورة، أو أسعار منزل).

W : الأوزان Weights

هذه هي إعدادات الآلة الداخلية كل مؤشر من مؤشراتك (X) لا يؤثر على مخرج واحد فقط، بل على كل المخرجات الممكنة بنسب مختلفة. هذه الإعدادات هي عقل الآلة أو ذاكرتها.

المعنى الحقيقي: هي مصفوفة الأرقام التي تتعلمها الشبكة العصبية. هي أهم جزء في الشبكة لأنها تحدد العلاقة بين المدخلات والمخرجات.

B : الانحيازات Biases

هذا هو التعديل الافتراضي أو موضع الخمول للآلة. بغض النظر عن كيف حركت المؤشرات، هناك دائماً قيمة أساسية تُضاف للمخرجات.

المعنى الحقيقي: هو متجه من الأرقام يسمح للشبكة بتحريك مخرجاتها للأعلى أو للأسفل، مما يمنحها مرونة أكبر.

4. الجزء الثاني: كيف تعمل الآلة ($Y = np.dot(X, W) + B$)

هذه هي المعادلة التي تصف عمل لوحة التحكم.

1. $np.dot(X, W)$

هذه هي العملية التي تقوم فيها الآلة بأخذ إعداداتها الداخلية (W) وتطبقها على مؤشرات التحكم (X) التي حركتها. كل مؤشر يؤثر على كل المخرجات بناءً على قيمته في مصفوفة الإعدادات. هذه هي عملية الخلط أو المعالجة الرئيسية.

2. $+ B$

بعد أن تقوم الآلة بالخلط، تضيف التعديل الافتراضي (B) إلى كل المخرجات كلمسة أخيرة.

5. الجزء الثالث: قاعدة الأبعاد (لماذا يجب أن تتطابق الأشكال؟)

$$\begin{array}{ccc} \mathbf{X} & \cdot & \mathbf{W} = \mathbf{O} \\ (2,) & & (2, 3) \quad (3,) \\ \hline & \text{Must be the same} & \end{array}$$

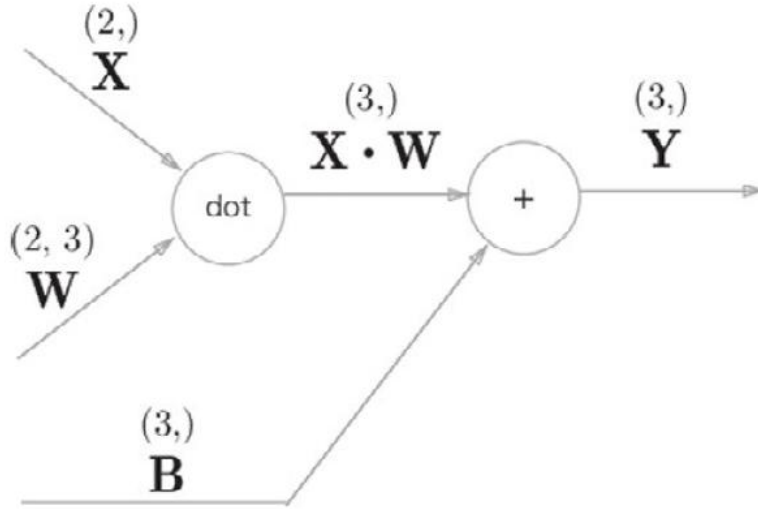
لا يمكنك ضرب أي مصفوفتين. يجب أن يكون هناك توافق بينهما.

- تخيل أن لديك قطعتين من الليغو تريد وصلهما. عدد النقاط البارزة في القطعة الأولى يجب أن يساوي عدد الثقوب في القطعة الثانية.
- في مثالنا:

- $X.shape$ مؤشرات للتحكم.
- $W.shape$ إعدادات الآلة. كل مخرج من المخرجات الثلاثة لديه إعدادان، واحد لكل مؤشر.
- التوافق: عدد المؤشرات (2) يطابق عدد الإعدادات لكل مخرج. هذا يعمل

ناتج ضرب المصفوفات في الانتشار الأمامي في الشبكة العصبية يُسمى affine transformation. لذلك، سنقوم بتنفيذ العملية التي تقوم بالتحويل الأفيني كـ Affine layer .

لنرى المرور الأمامي من خلال المخطط :



ملاحظة :

كيفية تطبيق مفهوم الانتشار الخلفي (Backpropagation) الذي شرحناه سابقاً والذي كان يعتمد على قيم فردية أو قيم عددية (scalar values) على المصفوفات المتعددة الأبعاد (Multidimensional Arrays)، وهي الأوزان والمدخلات الحقيقية في الشبكات العصبية لنشرح ذلك بالتفصيل :

طبقة Affine تحسب : $Y = XW + B$ حيث :

4. Y : مصفوفة المخرجات

5. X : مصفوفة المدخلات

6. W : مصفوفة الأوزان

7. B : متجه الانحيازات

1- الانتقال من القيم العددية إلى المصفوفات :

■ في الشرح السابق، تعلمنا الانتشار الخلفي باستخدام القيم الفردية (Scalars):

إذا كان $z = x \cdot y$ ضرب قيم عددية

في الانتشار الخلفي، فإن $\frac{\partial z}{\partial x} = y$

■ المشكلة الآن: في الشبكات العصبية، x و w ليستا قيمًا فردية، بل هما مصفوفات. الإخراج y هو أيضًا مصفوفة.

■ الحل: تخيل أنك تقوم بكتابة كل عنصر من عناصر المصفوفات وتطبيق قاعدة السلسلة على كل عنصر على حدة. ولكن هذه الطريقة غير عملية.

- **الطريقة العملية:** هي تطبيق قواعد اشتقاق المصفوفات مباشرة، والتي تشبه قواعد الاشتقاق العددية مع إضافة عمليات خاصة بالمصفوفات مثل المنقول (Transpose).

2- الانتشار الخلفي لعملية ضرب المصفوفات

لنركز على الجزء الأساسي $Y = XW$: لنفترض أن الإشارة الخلفية القادمة من اليمين (أي $\partial L / \partial Y$ تدرج الخسارة L بالنسبة لـ Y) نحتاج إلى حساب تدرجين:

1. التدرج بالنسبة للأوزان: أي $\frac{\partial L}{\partial W}$ هذا هو ما سنستخدمه لتعديل الأوزان.
2. التدرج بالنسبة للمدخلات: أي $\frac{\partial L}{\partial X}$ هذا هو ما سنمرره إلى الطبقة السابقة (اليسرى).

حساب التدرج بالنسبة للأوزان :

للحصول على تدرج الخسارة بالنسبة للأوزان W ، يتم ضرب الإشارة الخلفية $\frac{\partial L}{\partial Y}$ في منقول المدخلات X^T :

$$\frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Y}$$

لماذا منقول X :

هذا يعود إلى قواعد اشتقاق المصفوفات. عملية المنقول ضرورية لضمان توافق الأبعاد (Matrix Dimensions) لإجراء عملية الضرب التي تنتج مصفوفة $\frac{\partial L}{\partial W}$ بنفس أبعاد W

حساب التدرج بالنسبة للمدخلات :

للحصول على التدرج الذي سيمرر للطبقة السابقة يتم ضرب الإشارة الخلفية $\partial L / \partial Y$ في منقول الأوزان W^T :

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot W^T$$

3- شرح مفهوم المنقول (Transpose) :

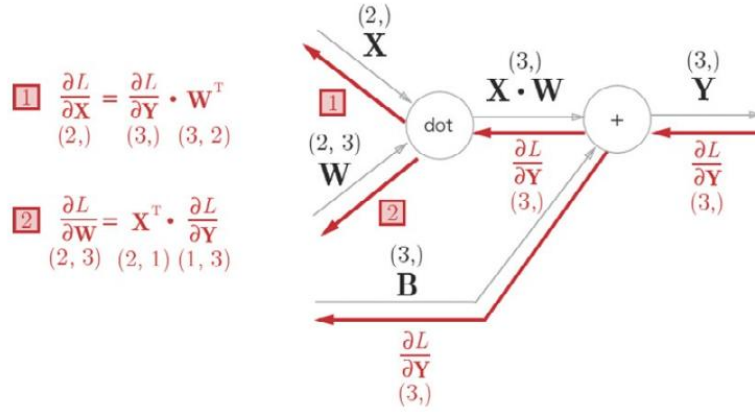
- ماهو المنقول :
- هو عملية تبديل أماكن الصفوف والأعمدة في المصفوفة
- إذا كانت W مصفوفة ذات الأبعاد (i, j) فإن W^T مصفوفة ذات الأبعاد (j, i)

$$W = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

$$W^T = \begin{pmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{pmatrix}$$

يجب استخدام عملية المنقول (Transpose) للمصفوفات X و W لضمان أن الضرب بين المصفوفات يحافظ على الأبعاد الصحيحة للتدرجات dX و dW

بناءً على المعادلات التي تعرفنا عليها لنرى المخططات الموافقة لها :



4- نلاحظ أن X و $\partial L / \partial X$ نفس الشكل

5- و W و $\partial L / \partial X$ نفس الشكل ايضا وذلك وفقا للمعادلات :

$$\mathbf{X} = (x_0, x_1, \dots, x_n)$$

$$\frac{\partial L}{\partial \mathbf{X}} = \left(\frac{\partial L}{\partial x_0}, \frac{\partial L}{\partial x_1}, \dots, \frac{\partial L}{\partial x_n} \right)$$

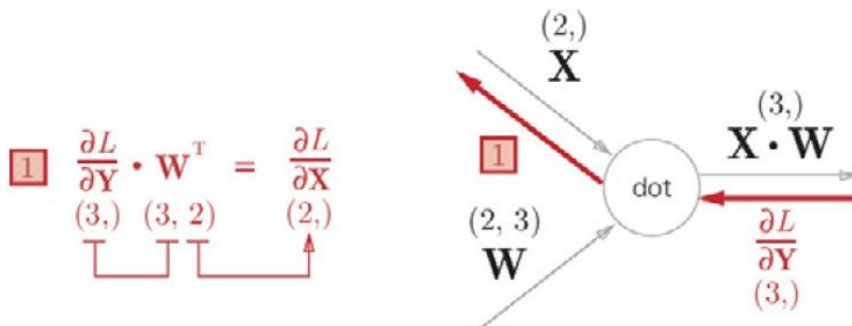
6- لماذا يجب ان يكونوا من نفس الشكل ؟

لان التدرج $\partial L / \partial X$ يمثل معدل تغير الخسارة لكل عنصر من عناصر X وبما أن لكل عنصر في X يجب أن يقابله تدرج واحد فقط، فيجب أن يكون شكل المصفوفتين متطابقاً حيث X مصفوفة الدخل في الانتشار الامامي و $\partial L / \partial X$ مصفوفة الخرج في الانتشار الخلفي

إذاً :

نحن ننتبه لأشكال المصفوفات لأن عدد العناصر في الأبعاد المقابلة يجب أن يكون متساوياً من أجل ضرب المصفوفات.

كما في المخطط التالي :



من اجل كل عقدة في المخطط يجب ان يكون :

1. شكل مخرج الانتشار الأمامي يجب أن يساوي شكل مُدخل الانتشار الخلفي .
2. شكل مُدخل الانتشار الأمامي يجب أن يساوي شكل مخرج الانتشار الخلفي .

Batch-Based Affine Layer

مثال لفهم الفكرة :

المقدمة: من صنع ساندوتش واحد إلى طلب جماعي

تخيل أن طبقة الأفين هي محطة صنع ساندويتش

- سابقاً (بدون تجميع) : كان الزبون يأتي ويعطيك مكونين (X بحجم (2,) ، وأنت تأخذهما، تضعهما في الخلاط (W)، وتضيف لهما التوابل (B) لتصنع له ساندويتشاً واحداً (Y بحجم (3,)) ثم يأتي الزبون التالي... وهكذا هذه الطريقة بطيئة وغير فعالة.
- الآن (مع التجميع Batch) : جاءك طلب جماعي من N زبون في نفس الوقت. كل زبون أعطاك مكونين بدلاً من صنع الساندويتشات واحدة تلو الأخرى، وضعت كل مكونات الطلبات في صواني كبيرة.
- هذا هو التجميع : معالجة مجموعة من البيانات N عينة في نفس الوقت، بدلاً من عينة واحدة. هذا أسرع بآلاف المرات باستخدام المعالجات الحديثة.

1. الجزء الأول: المرور الأمامي (Forward Pass) صنع الساندويتشات دفعة واحدة

الآن، لنرى كيف يتغير شكل المكونات في المصنع.

1- المدخلات (المكونات) :

- X مكونات الزبائن : لم تعد متجهاً واحداً (2,) أصبحت الآن مصفوفة كبيرة تحتوي على مكونات كل N زبون. شكلها هو ($N, 2$)
- N : عدد الزبائن (حجم الدفعة).
- 2 : عدد المكونات لكل زبون.
- W الخلاط : لم يتغير هو نفس الخلاط الذي نستخدمه لكل الساندويتشات شكله (3, 2)
- B التوابل : لم يتغير نفس كمية التوابل نضيفها لكل ساندوتش شكله (3,)

2- العملية الحسابية ($Y = np.dot(X, W) + B$) :

$A = np.dot(X, W)$:

- نضرب مصفوفة المكونات ($N, 2$) في مصفوفة الخلاط (3, 2)
- النتيجة هي مصفوفة المخرجات الأولية Y_temp بشكل ($N, 3$)
- ماذا يعني هذا؟ كل صف في Y_temp يمثل الساندويتش شبه النهائي لزبون معين لقد قمنا بعملية الخلط لكل الزبائن دفعة واحدة

B. + :

- لدينا Y_temp بحجم (N, 3) و B بحجم (3,) كيف يمكننا جمع مصفوفة ومتجه؟
 - هنا يقوم NumPy بعملية تسمى (Broadcasting)
 - تشبيه: تخيل أن B هي ختم بشكل (3,) يقوم NumPy تلقائياً بأخذ هذا الختم وطبعه على كل صف من صفوف Y_temp
 - بمعنى آخر، يتم إضافة متجه B إلى كل صف من Y_temp بشكل منفصل
- النتيجة النهائية للمرور الأمامي:** نحصل على مصفوفة Y نهائية بحجم (N, 3) كل صف i في هذه المصفوفة هو المخرج النهائي Y_i للمدخل X_i.

2. الجزء الثاني: المرور الخلفي (Backward Pass) تقييم الأخطاء والتحسين

الآن، جاءت النتائج، والزبائن أعطونا ملاحظاتهم (التدرج dout) شكل ملاحظاتهم dout هو (N, 3)، حيث كل صف يحتوي على تقييم الساندوتش الخاص بهم. مهمتنا هي معرفة كيف نصلح الخلط W والتوابل B بناءً على هذه الملاحظات.

1- حساب تدرج المدخلات (dx):

الهدف: معرفة كيف أثر كل مكون أصلي في X على الخطأ النهائي.

المعادلة: $dx = np.dot(dout, W^T)$

- **الشرح:** لنعكس عملية الضرب، نحتاج إلى مسار عكسي. هذا المسار هو مصفوفة W المنقلبة (W^T) بشكل (3,2)

■ التحقق من الأشكال:

W^T شكلها (3, 2)

الآن نضرب dout شكلها (N, 3) ب W^T

نتيجة الضرب تكون (N, 2)

إذاً، شكل المخرج dx هو (N, 2)

قاعدتنا تقول شكل dx (المخرج من المرور الخلفي) يجب أن يساوي شكل X (المدخل للمرور الأمامي) شكل dx هو (N, 2) وشكل X هو (N, 2) إذاً يوجد تطابق.

2- حساب تدرج الأوزان (dw):

الهدف: معرفة كيف يجب تعديل الخلط W بناءً على أخطاء جميع الزبائن مجتمعين.

المعادلة: $dw = np.dot(X^T, dout)$

- **الشرح:** هنا ننظر إلى كيفية تأثير كل وزن في W على الخطأ الكلي. لذلك نأخذ المدخلات الأصلية X ونقلبها X^T لتصبح (2, N) ونضربها في ملاحظات الزبائن dout (N, 3) هذه العملية تجمع تأثير كل عينة من الـ N عينة على كل وزن في W
- **التحقق من الأشكال:**

X^T شكلها (2, N)dout شكلها (N, 3) نضربها بـ X^T

شكل المخرج dw هو (2, 3)

قاعدتنا تقول شكل dw (المخرج من المرور الخلفي) يجب أن يساوي شكل W (المدخل للمرور الأمامي)

شكل dw هو (2, 3) وشكل W هو (2, 3) إذًا يوجد تطابق

3- حساب تدرج الانحيازات (dB):

الهدف: معرفة كيف يجب تعديل التوابل B

المعادلة: $dB = np.sum(dout, axis=0)$

- الشرح: تذكر تشبيه الختم تم إضافة نفس B إلى كل صف من Y_temp لذلك، فإن الخطأ الناتج من كل صف (dout) يجب أن يدفع على هذا الختم. التأثير الكلي على الختم هو مجموع كل هذه الدفعات.
 - $np.sum(dout, axis=0)$ يعني: اجمع كل الأعمدة هذا يجمع تأثير الخطأ من كل N عينة لكل من الـ 3 توابل.
 - التحقق من الأشكال $dout(N, 3)$: يتم جمعها على طول الأعمدة ليعطينا $dB(3,)$
- النتيجة: الشكل (3,) يطابق شكل B الأصلي

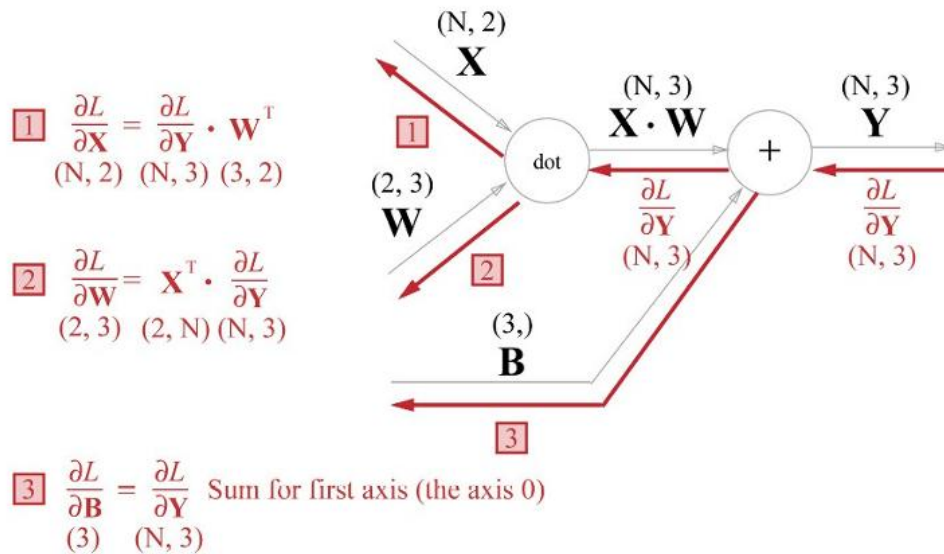
شكل الخرج	أشكال المدخلات	المعادلة	العملية
$Y(N,3)$	$X(N,2), W(2,3), B(3,)$	$Y = X \cdot W + B$	(Forward)
$dX(N,2)$	$dout(N,3), W^T(3,2)$	$dX = dout \cdot W^T$	(Backward)
$dW(2,3)$	$X^T(2,N), dout(N,3)$	$dW = X^T \cdot dout$	(Backward)
$dB(3,)$	$dout(N,3)$	$dB = np.sum(dout, axis=0)$	(Backward)

Work hard in silence,

Let your success be your noise



لنرى المخطط الذي يعبر عن هذا المفهوم :



مثال عملي :

1. الجزء الأول: الانتشار الأمامي

```
121 X_dot_W = np.array([[0, 0, 0], [10, 10, 10]])
122 B = np.array([1, 2, 3])
123 print(X_dot_W + B)
```

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[[ 1  2  3]
 [11 12 13]]
```

الشرح :

السؤال : كيف يمكننا جمع مصفوفة بحجم (2x3) مع متجه بحجم (3,) رياضياً، هذا غير ممكن مباشرة

الإجابة : نستخدم (Broadcasting)

شرح مبسط : كما ذكرنا قبل قليل مثال الختم عند جمع B مع كل صف من صفوف X_dot_W تصبح العملية كالتالي :

- تأخذ الختم B: [1, 2, 3]
- تكرر ليتطابق شكله مع شكل X_dot_W في هذه الحالة، ينسخ B لإنشاء مصفوفة جديدة بحجم (3, 2) حيث كل صف هو نسخة من B
- نقوم بعملية الجمع عنصر بعنصر (Element-wise):
الصف الأول: $[3, 2, 1] = [3, 2, 1] + [0, 0, 0]$
الصف الثاني: $[13, 12, 11] = [3, 2, 1] + [10, 10, 10]$
- الخلاصة: في الانتشار الأمامي، تم استخدام نفس متجه الانحياز B وإضافته إلى كل عينة بيانات (كل صف في X_dot_W)

```

125 dY = np.array([[1, 2, 3,], [4, 5, 6]])
126 dB = np.sum(dY, axis=0)
127 print(dB)

```

PROBLEMS 10

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

[5 7 9]

الشرح :

حساب التدرج (Gradient) للانحياز B في عملية الانتشار الخلفي

السؤال : لماذا قمنا بجمع عناصر dY على طول الأعمدة (axis=0) لنحصل على dB؟

الإجابة : هذا يعود مباشرة إلى ما حدث في الانتشار الأمامي.

المنطق:

1- في الانتشار الأمامي، العنصر الأول من B وهو 1 أضيف إلى العنصر الأول من الصف الأول (0) و إلى العنصر الأول من الصف الثاني (10)

2- في الانتشار الخلفي، يصلنا تدرج الخطأ dY هذا التدرج يخبرنا كيف أثر كل مخرج على الخطأ الكلي.

- $dY[0, 0]$ (قيمه 1) هو تدرج الخطأ بالنسبة للمخرج الأول للعينة الأولى.

- $dY[1, 0]$ (قيمه 4) هو تدرج الخطأ بالنسبة للمخرج الأول للعينة الثانية.

3- بما أن نفس العنصر $B[0]$ أثر على كلا المخرجين، فإن التدرج الكلي $dB[0]$ يجب أن يكون مجموع تأثيره على كل منهما.

لنطبق هذا المنطق على كل عنصر في B :

- لحساب $dB[0]$ التدرج للعنصر الأول في B :

- يجب أن نجمع التدرجات التي أثر عليها، وهي موجودة في العمود الأول من dY

- $dB[0] = dY[0, 0] + dY[1, 0] = 1 + 4 = 5$

- لحساب $dB[1]$ التدرج للعنصر الثاني في B :

- نجمع التدرجات من العمود الثاني في dY

- $dB[1] = dY[0, 1] + dY[1, 1] = 2 + 5 = 7$

- لحساب $dB[2]$ التدرج للعنصر الثالث في B :

- نجمع التدرجات من العمود الثالث في dY

- $dB[2] = dY[0, 2] + dY[1, 2] = 3 + 6 = 9$

هذا بالضبط ما تفعله الدالة $np.sum(dY, axis=0)$

$axis=0$ تعني اجمع على طول المحور العمودي (أي اجمع كل الصفوف معاً لكل عمود على حدة).

خلاصة:

لأنه في الانتشار الأمامي تم مشاركة نفس متجه الانحياز B بين جميع عينات الدفعة (Batch)، فإنه في الانتشار الخلفي

يجب جمع تدرجات الخطأ من جميع هذه العينات لحساب التدرج النهائي لـ B .

the implementation of the Affine layer : class Affine

```

129 class Affine:
130     def __init__(self, W, b):
131         self.W = W
132         self.b = b
133         self.x = None
134         self.dW = None
135         self.db = None
136     def forward(self, x):
137         self.x = x
138         out = np.dot(x, self.W) + self.b
139         return out
140     def backward(self, dout):
141         dx = np.dot(dout, self.W.T)
142         self.dW = np.dot(self.x.T, dout)
143         self.db = np.sum(dout, axis=0)
144         return dx

```

الشرح :

1. أولاً: مرحلة الإعداد والمواصفات (`__init__`) : فكر فيها ك تصميم المصنع

- `: self.W = W`
نعطي المصنع آلة الخلط الخاصة به (مصفوفة الأوزان)
- `: self.b = b`
نعطيه مقابض الضبط (متجه الانحيازات)
- `: self.x = None`
هذا هو الجزء الأهم. نجهز لوحة ملاحظات للمصنع لتسجيل المدخلات التي استلمها في البداية اللوحة فارغة
- `: self.dW = None, self.db = None`
نجهز لوحتين ملاحظات أخريين لتسجيل نتائج تحسين الأداء (التدرجات) التي سيحسبها المصنع لاحقاً
- الخلاصة: في هذه المرحلة، نعطي الطبقة أدواتها (`W, b`) ونجهز ذاكرتها (`x, dW, db`) لتكون جاهزة للعمل .

2. ثانياً: مرحلة العمل والإنتاج (`forward`)

هذه الدالة تمثل الانتشار الأمامي، أي عندما تمر البيانات عبر الطبقة من اليسار إلى اليمين.

- `: self.x = x`
أول شيء يفعله المصنع عند استلام المواد الخام (`x`) هو أنه يسجلها فوراً على لوحة الملاحظات (`self.x`)

- لماذا؟ لأنه سيحتاج إلى معرفة المواد الخام الأصلية بالضبط عندما يريد حساب كيفية تحسين أدائه في مرحلة backward هذه هي خطوة التذكر الحاسمة .
- `: out = np.dot(x, self.W) + self.b`
- هنا يتم العمل الحقيقي يتم خلط المواد الخام (x) باستخدام آلة الخلط (self.W) وإضافة اللمسة النهائية (self.b) هذه هي معادلة $Y = X \cdot W + B$ التي نعرفها
- `: return out`
- يسلم المصنع المنتج النهائي (out) إلى المحطة التالية في خط الإنتاج (الطبقة التالية في الشبكة)

3. ثالثاً: مرحلة التعلم وتحسين الأداء (backward)

- هذه الدورة تمثل الانتشار الخلفي، وهي أهم مرحلة للتعلم. هنا، يحلل المصنع تقرير الجودة (dout) القادم من المحطة التالية ليحدد كيف يمكنه تحسين أدائه.
- `dout` : هذا هو تقرير الخطأ أو التدرج الذي يخبر المصنع مدى مساهمة مخرجاته (out) في الخطأ الكلي.
- `: dx = np.dot(dout, self.W.T)`
- الهدف: حساب الخطأ الخاص بالمدخلات (x)
- كيف؟ لنعرف كيف أثرت المواد الخام (x) على الخطأ، نمرر تقرير الخطأ (dout) عبر آلة الخلط في الوضع العكسي (self.W.T)
- النتيجة dx: هو تدرج الخطأ الذي سيتم إرساله إلى المحطة السابقة.
- `: self.dW = np.dot(self.x.T, dout)`
- الهدف: حساب كيف يجب تعديل آلة الخلط (W)
- كيف؟ لتحسين الآلة (W)، نقارن بين المواد الخام الأصلية التي سجلناها (self.x) وتقرير الخطأ النهائي (dout)
- النتيجة self.dW: يُخزن في ذاكرة المصنع. هذا هو التحديث المطلوب للأوزان W
- `: self.db = np.sum(dout, axis=0)`
- الهدف: حساب كيف يجب تعديل مقابض الضبط (b)
- كيف؟ بما أن نفس مقبض الضبط (b) أضيف لكل منتج، فإن الخطأ الكلي له هو مجموع الأخطاء من كل المنتجات
- النتيجة self.db: يُخزن في ذاكرة المصنع. هذا هو التحديث المطلوب للانحيازات b
- `: return dx`
- يرسل المصنع تقرير الخطأ الخاص بالمدخلات (dx) إلى المحطة السابقة في خط الإنتاج، حتى تقوم هي الأخرى بتحسين أدائها.

المتغير	وظيفته	متى يستخدم
self.W	آلة الخلط (الأوزان)	في forward للخلط، وفي backward لحساب dx
self.b	مقابض الضبط (الانحيازات)	في forward للإضافة
self.x	ذاكرة المدخلات	يُسجل في forward، ويُستخدم في backward لحساب self.dW
self.dW	ذاكرة تحسين الأوزان	يُحسب في backward ليتم استخدامه لتحديث self.W
self.db	ذاكرة تحسين الانحيازات	يُحسب في backward ليتم استخدامه لتحديث self.b

Softmax-with-Loss Layer

مثال لفهم الفكرة :

فكر في طبقة Softmax-with-Loss على أنها القاضي النهائي ومحلل الأداء في الشبكة العصبية دعنا نقسم الشرح إلى جزأين رئيسيين لفهمهما بشكل كامل :

1. ماذا تفعل طبقة Softmax ؟ (دور القاضي)
2. لماذا ندمجها مع دالة الخسارة (Loss) ؟ (دور محلل الأداء)

1. الجزء الأول: طبقة Softmax القاضي الذي يحول النقاط إلى احتمالات

- تخيل أن الشبكة العصبية هي لجنة تحكيم في مسابقة للتعرف على الأرقام المكتوبة بخط اليد (من 0 إلى 9).
- المدخلات (قبل القاضي) :** قبل أن تصل النتائج إلى القاضي النهائي (Softmax)، الطبقة الأخيرة من الشبكة (طبقة Affine) تخرج 10 أرقام عشوائية. هذه الأرقام تسمى النقاط (Scores) أو "الدرجات هي مجرد مؤشرات خام لثقة الشبكة.
- مثال :** قد تخرج الشبكة هذه النقاط للصورة المدخلة : [-1.2, 5.1, 0.8, 3.2, ...]
- المشكلة :** هذه الأرقام ليس لها معنى مباشر.
- أي رقم يعني الأعلى ثقة؟ هل 5.1 تعني ثقة 50% أم 90%؟ لا نعرف.**
- دور طبقة Softmax القاضي :** تأتي طبقة Softmax لتحل هذه المشكلة. وظيفتها هي أن تأخذ هذه النقاط الخام وتحولها إلى احتمالات مفهومة.
- العملية :** تقوم بعملية تطبيع (Normalization) هذا يعني أنها تحول جميع النقاط إلى نسب مئوية، بحيث يكون مجموعها الإجمالي يساوي 1 (أو 100%).

■ **المخرجات (بعد القاضي):** النقاط السابقة [1.2, 5.1, 0.8, 3.2, ...] تتحول إلى احتمالات مثل: [0.01, 0.85, 0.02, 0.10, ...]

■ **المعنى:** الآن أصبحت النتائج واضحة جداً الشبكة تقول: أنا متأكدة بنسبة 85% أن الرقم في الصورة هو '1'، ومتأكدة بنسبة 10% أنه '3'، وهكذا.

الخلاصة: طبقة Softmax هي مترجم يترجم لغة الأرقام الخام (النقاط) إلى لغة مفهومة (الاحتمالات).

2. الجزء الثاني: التمييز بين الاستنتاج (Inference) والتدريب (Training)

لماذا نحتاج لهذا القاضي في مرحلة ما، ولا نحتاجه في مرحلة أخرى؟

الحالة الأولى: الاستنتاج (Inference) هدفنا هو الإجابة السريعة

- **الهدف:** عندما نريد من الشبكة أن تخمن إجابة (مثل تطبيق يتعرف على الأرقام من الكاميرا)، كل ما نريده هو إجابة واحدة صحيحة وأسرع وقت ممكن.
- **الطريقة:** لمعرفة الإجابة، هل نحتاج لكل هذه الاحتمالات المعقدة؟ لا! كل ما نحتاجه هو معرفة أي الرقم حصل على أعلى نقطة من طبقة Affine مباشرة.
- **التشبيه:** تخيل سباقاً. لمعرفة الفائز، هل تحتاج إلى حساب الفروق الزمنية الدقيقة بين كل المتسابقين؟ لا، يكفي أن تنظر إلى من خط في النهاية أولاً.
- **النتيجة:** في مرحلة الاستنتاج، طبقة Softmax هي خطوة إضافية غير ضرورية وتبطئ العملية. نحن نأخذ النقاط الخام من طبقة Affine ونبحث عن القيمة الأعلى مباشرة باستخدام دالة مثل argmax

الحالة الثانية: التدريب (Training) هدفنا هو التعلم من الأخطاء

- **الهدف:** عندما نقوم بتدريب الشبكة، نريد أن نتعلم ونتحسن. التعلم لا يحدث بمعرفة الإجابة الصحيحة فقط، بل بمعرفة مدى الخطأ في جميع الإجابات الأخرى.
- **المشكلة:** إذا قلنا للشبكة الإجابة الخاطئة هي '1'، كيف تعرف كم كانت مخطئة؟ هل كانت احتمالاتها للإجابة '3' قريبة جداً أم بعيدة جداً؟ النقاط الخام لا تخبرنا بذلك.
- **الحل:** هنا يأتي دور طبقة Softmax-with-Loss

1. Softmax : تحول النقاط إلى احتمالات [0.01, 0.85, 0.02, 0.10, ...]

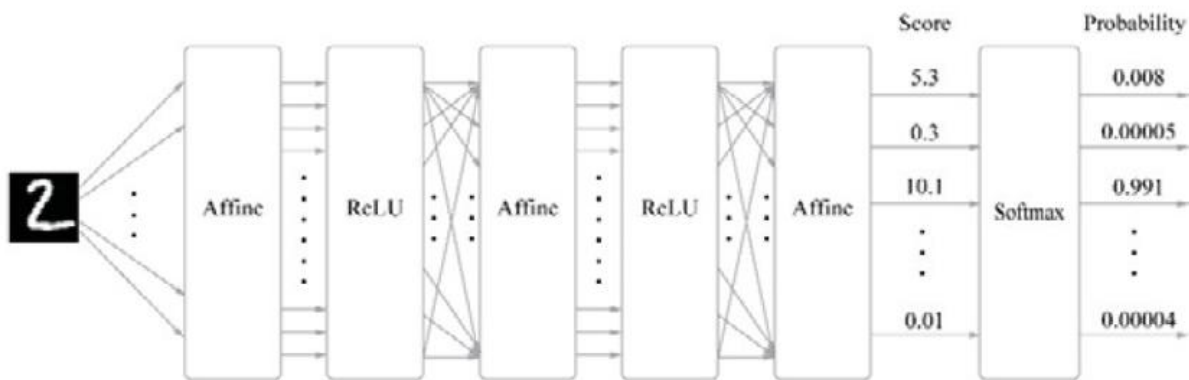
2. Loss Function دالة الخسارة: هذه هي محلل الأداء. تأخذ الاحتمالات التي خرجت من Softmax وتقارنها بالإجابة الصحيحة (مثلاً، الرقم الصحيح هو '3').

3. المقارنة تنتج لنا رقماً واحداً يسمى الخسارة (Loss) هذا الرقم يقيس مدى سوء أداء الشبكة. كلما كانت الاحتمالات الخاطئة عالية، زادت قيمة الخسارة.

- **النتيجة:** هذه القيمة (الخسارة) هي التي توجه عملية التعلم. هي التي تخبر الشبكة: أنت مخطئة جداً، ويجب عليك تعديل أوزانك بشكل كبير. بدون Softmax، لا يمكننا حساب هذه الخسارة بدقة، وبالتالي لا يمكن للشبكة أن تتعلم بفعالية.

المرحلة	الهدف	هل نستخدم softmax ؟	لماذا
الاستنتاج (Inference)	الحصول على إجابة سريعة ودقيقة	لا	نحتاج فقط لأعلى نقطة (score) وليس للاحتمالات. إنها خطوة غير ضرورية
التدريب (Training)	تعلم الشبكة وتحسين أدائها من أخطائها	نعم، ضرورية جداً	نحتاج للاحتمالات لحساب "الخسارة (Loss)"، وهي المقياس الذي يخبر الشبكة بمدى خطئها وكيفية التحسن

مثال :



الشرح :

كما تحدثنا فكر في طبقة Softmax-with-Loss على أنها القاضي النهائي ومصدر شهادة التقدير في مدرسة الشبكات العصبية.

دعنا نقسم الشرح إلى أجزاء منظمة وواضحة :

أولاً: المرور الأمامي (Forward Pass) إصدار الحكم والشهادة

هذه العملية تتكون من خطوتين متتاليتين:

الخطوة 1: طبقة (Softmax القاضي يصدر حكمه)

- المدخلات: تستقبل الطبقة النقاط الخام من الطبقة السابقة (Affine Layer) في المثال، النقاط هي 5.3 للرقم 0 و 10.1 للرقم 2.
- المهمة: تحويل هذه النقاط العشوائية إلى احتمالات مفهومة ومقنعة.
- العملية: تقوم بعملية تطبيع، بحيث يصبح مجموع كل الاحتمالات 100%

■ المخرجات (الحكم) :

- النقطة 5.3 تتحول إلى احتمال 0.008 أي 0.8%
- النقطة 10.1 تتحول إلى احتمال 0.991 أي 99.1%
- الآن الحكم واضح: الشبكة متأكدة بنسبة 99.1% أن الرقم هو "2"

الخطوة 2: طبقة الخطأ التقاطعي (Cross-Entropy Error) (كتابة شهادة التقدير)

- المدخلات: تستقبل حكم القاضي الاحتمالات (y) و تستقبل الإجابة الصحيحة (t)
 - المهمة: مقارنة حكم القاضي بالحقبة المطلقة، وإعطاء درجة نهائية واحدة تسمى "الخسارة (Loss)" هذه الدرجة تقيس مدى سوء أداء الشبكة.
 - القاعدة: كلما كانت احتمالات الشبكة للإجابة الخاطئة عالية، زادت درجة الخسارة (وساء أداء الشبكة).
- الخلاصة: في نهاية المرور الأمامي، نحصل على رقم واحد الخسارة لا يخبرنا بمدى بعد الشبكة عن الإجابة الصحيحة.

ثانياً: المرور الخلفي (Backward Pass) أهمية شهادة التقدير

النتيجة :

عندما نرسل إشارة الخطأ للخلف من خلال طبقة Softmax-with-Loss ، فإن النتيجة ليست معقدة أبداً. إنها ببساطة:

$$(y_1 - t_1, y_2 - t_2, y_3 - t_3)$$

- ماذا يعني هذا؟ إنه ببساطة الفرق أو الخطأ بين ما توقعت الشبكة (y) وما هو صحيح في الواقع (t)
 - y: هي احتمالات الشبكة (مثلاً (0.3, 0.2, 0.5))
 - t: هي الإجابة الصحيحة (مثلاً (0, 1, 0))
 - y - t: هو الخطأ الفعلي (مثلاً (0.3, -0.8, 0.5))
 - لماذا هذه النتيجة مهمة جداً؟
- هذه النتيجة ليست صدفة! لقد تم اختيار دالة الخطأ التقاطعي (Cross-Entropy Error) عن قصد لتعمل بشكل مثالي مع دالة Softmax وتعطينا هذا التعبير البسيط والواضح عن الخطأ.
- لماذا هذا التعبير مهم؟ لأنه يخبر الطبقات السابقة بالضبط ما يجب عليها فعله:
 - إذا كان الخطأ كبيراً: يجب على الطبقات السابقة أن تقوم بتعديل كبير في أوزانها.
 - إذا كان الخطأ صغيراً: يكفيها تعديل بسيط جداً.

دعنا نرى هذا في مثالين:

المثال الأول: الشبكة تخطئ خطأ فادحاً

- الإجابة الصحيحة (t) : (0, 1, 0) (الرقم هو "1")
- توقع الشبكة (y): (0.3, 0.2, 0.5) الشبكة هنا سيئة جداً، فهي تعتقد أن الإجابة الصحيحة هي "2" باحتمال 50%، وأن "1" هو الإجابة الصحيحة باحتمال 20% فقط!
- الخطأ (y - t) :

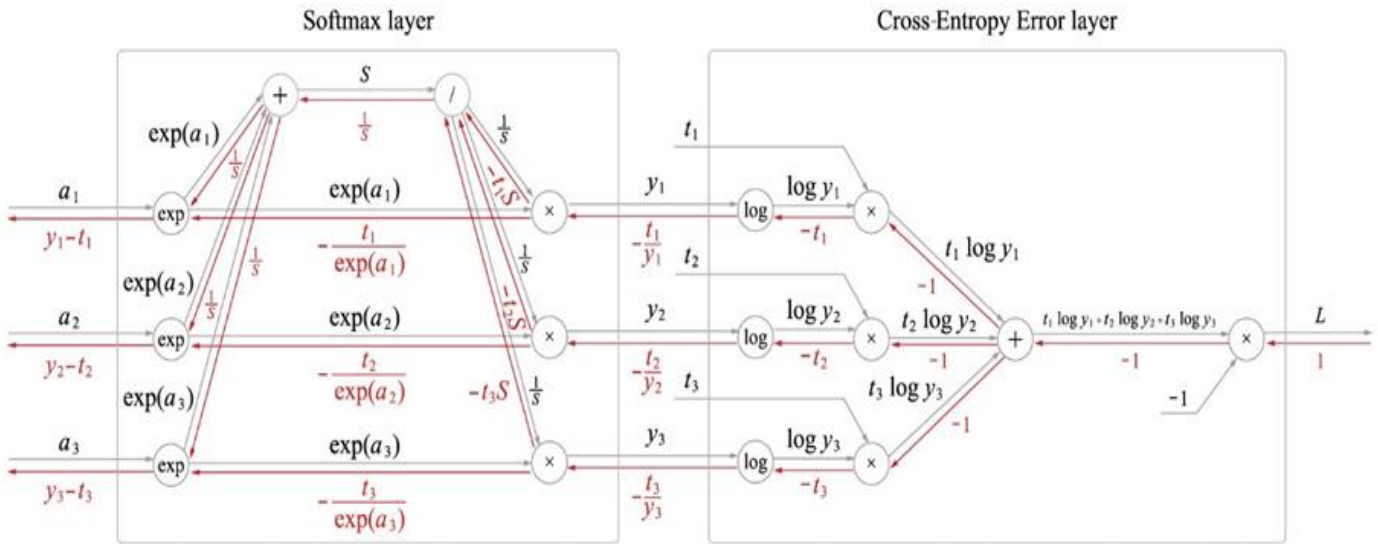
- $(0.3, 0.8, 0.5) = (0.3 - 0, 0.8 - 1, 0.5 - 0)$
- التحليل: الرقم 0.8 هو خطأ كبير جداً! هذه الإشارة القوية ترسل إلى الطبقات السابقة وكأنها إنذار حريق، تخبرهم: أنتم أخطأتم بشدة في الخلية الثانية، قوموا بتعديل أوزانكم بشكل كبير وسريع!

المثال الثاني: الشبكة تتوقع بشكل جيد جداً

- الإجابة الصحيحة (t): (0, 1, 0)
- توقع الشبكة (y): (0.01, 0.99, 0)
- الخطأ (y - t):
- $(0.01, -0.01, 0) = (0.01 - 0, 0.99 - 1, 0 - 0)$
- التحليل: الرقم 0.01 هو خطأ صغير جداً. هذه الإشارة الضعيفة ترسل إلى الطبقات السابقة وكأنها همسة لطيفة، تخبرهم: أداؤكم رائع، ربما تحتاجون لتعديل طفيف جداً لا يذكر

خلاصة:

طبقة Softmax-with-Loss هي تصميم عبقرى لأنها تحول مشكلة التعلم المعقدة إلى إشارة خطأ بسيطة وواضحة (الفرق بين التوقع والواقع). هذا يسمح للشبكة بتقييم أدائها بدقة وإرسال ملاحظات دقيقة إلى طبقاتها لتتعلم وتتحسن بشكل فعال. لنرى المخطط:



علماً أن المخطط مشروح بالتفصيل في ملف أرسل من قبل المعيد .

implement the Softmax-with-Loss layer

```

158 class SoftmaxWithLoss:
159     def __init__(self):
160         self.loss = None # Loss
161         self.y = None # Output of softmax
162         self.t = None # Label data (one-hot vector)
163     def forward(self, x, t):
164         self.t = t
165         self.y = softmax(x)
166         self.loss = cross_entropy_error(self.y, self.t)
167         return self.loss
168
169     def backward(self, dout=1):
170         batch_size = self.t.shape[0]
171         dx = (self.y - self.t) / batch_size
172         return dx

```

الشرح :

فكر في هذا الكود على أنه القاضي النهائي الذي لا يصدر حكماً فحسب، بل يكتب تقريراً مفصلاً جداً عن كيفية تحسين الأداء

دعنا نقسم الشرح إلى ثلاثة أجزاء منظمة:

الجزء الأول: تفكيك كود القاضي (SoftmaxWithLoss class)

1. مرحلة الإعداد (init)

هنا، نجهز ذاكرة للقاضي.

- `self.y` : سيحتفظ باحتمالاته (مثلاً (0.01, 0.99, 0))
- `self.t` : سيحتفظ بالإجابة الصحيحة (مثلاً (0, 1, 0))
- `self.loss` : سيحتفظ بالدرجة النهائية (الخسارة)

2. مرحلة إصدار الحكم (forward)

هذه هي العملية الأمامية التي نعرفها :

- `softmax(x)` : يأخذ النقاط الخام x ويحولها إلى احتمالات
- `cross_entropy_error(self.y, self.t)` : يقارن الاحتمالات y بالإجابة الصحيحة t ويحسب درجة الخسارة
- الأهم : يخزن الاحتمالات y والإجابة t في ذاكرته (`self.y, self.t`) لاستخدامها لاحقاً.

3. مرحلة كتابة تقرير التحسين (backward)

هذه هي عملية الانتشار الخلفي.

- $(self.y - self.t)$:
- هذا هو الخطأ أو الفرق الذي تحدثنا عنه. إنه ببساطة الفرق بين توقع الشبكة والواقع.
- $batch_size$ / :
- هذه هي خطوة التطبيع أو التوحيد القياسي

لماذا نقسم على عدد العينات؟

- تخيل أنك تدرس 10 طلاب مجموع درجاتهم الكلي هو 850.
- هل هذا يعني أن كل طالب حصل على 850؟ طبعاً لا
- لمعرفة متوسط أداء الطالب، يجب أن تقسم على عدد الطلاب ($850 / 10 = 85$)
- نفس المنطق هنا $(self.y - self.t)$: يعطينا مجموع الخطأ من كل عينة في الدفعة.
- لنحصل على متوسط الخطأ لكل عينة، نقسم على حجم الدفعة ($batch_size$) هذا يضمن أن حجم التحديث لا يتغير بشكل عشوائي فقط لأننا غيرنا حجم الدفع.

الخلاصة: تمرير backward يرسل إشارة واضحة ومتوازنة إلى الطبقات السابقة تقول: متوسط خطأكم كان كذا،

قوموا بتعديل أنفسكم بناءً على هذا المتوسط .

Implementing Backpropagation

يمكننا بناء شبكة عصبية كاملة عن طريق دمج هذه الطبقات كما لو كنا نجمع قطع الليغو.

- كل طبقة (Affine, ReLU, SoftmaxWithLoss) هي قطعة ليغو مستقلة.
- كل قطعة لها واجهة قياسية: منفذ forward للدخول ومنفذ backward للخروج
- لبناء شبكة، كل ما عليك فعله هو ترتيب هذه القطع بالترتيب الذي تريده وربطها ببعضها البعض
- مثال بسيط لشبكة:
المدخلات <- [Affine] <- [ReLU] <- [Affine] <- [SoftmaxWithLoss] <- الخسارة

Overall View of Neural Network Training

الصورة الكاملة لتدريب الشبكة العصبية

الآن، لنضع كل شيء في سياقه. كيف نستخدم كل هذه القطع لتدريب شبكة؟ العملية تتكرر باستمرار وتسمى حلقة التدريب (Training Loop)

1. الحصول على دفعة من البيانات (Get a Mini-Batch): نختار مجموعة صغيرة عشوائياً من بيانات التدريب (مثلاً 100 صورة).

2. المرور الأمامي (Forward Pass): نمرر هذه الدفعة عبر الشبكة قطعة قطعة.
البيانات -> Affine -> ReLU -> Affine -> SoftmaxWithLoss

في النهاية، نحصل على قيمة الخسارة (Loss)

3. **المرور الخلفي (Backward Pass):** نبدأ من النهاية ونمرر إشارة الخطأ للخلف.

SoftmaxWithLoss -> Affine -> ReLU -> Affine

كل طبقة تحسب تدرجاتها الخاصة (dW, db) وتحسب التدرج الذي سترسله للطبقة التي قبلها.

4. **تحديث الأوزان (Update Weights):** نستخدم التدرجات التي حسبتها (dW, db) لتحديث أوزان الشبكة

قليلاً في الاتجاه الصحيح باستخدام خوارزمية مثل SGD

5. **التكرار : (Repeat)** نكرر هذه العملية آلاف أو ملايين المرات، مع كل مرة نأخذ دفعة جديدة من البيانات، حتى تصبح الشبكة دقيقة جداً.

Presupposition

توضيح الفكرة من خلال مثال :

الشيف الطباخ الذي يحاول إتقان وصفة جديدة

المقدمة: الفكرة الأساسية (Presupposition)

4- تخيل أن الشبكة العصبية هي شيف طباخ، والأوزان والانحيازات (Weights and Biases) هي **مقادير**

الوصفة التي يستخدمها (كمية الملح، السكر، التوابل، وقت الطهي...).

5- في البداية، الشيف لا يعرف المقادير المثالية، لذا يبدأ بمقادير عشوائية.

6- عملية التدريب (Training) هي العملية التي يتبعها الشيف لتعديل هذه المقادير تدريجياً حتى تصبح الوجبة مثالية

7- الهدف هو تعديل الوصفة (الأوزان والانحيازات) لتناسب مع بيانات التدريب (ذوق العملاء المثالي).

الخطوات الأربع لإتقان الوصفة (تدريب الشبكة العصبية)

هذه هي العملية التي يتبعها الشيف (الشبكة) مراراً وتكراراً:

الخطوة 1: اختيار دفعة صغيرة (Mini-Batch)

■ **في عالم الشيف:** الشيف لا يتذوق الطبق الضخم بأكمله في كل مرة. هذا سيستغرق وقتاً طويلاً وقد لا يكون دقيقاً. بدلاً من ذلك، يأخذ **ملعقة صغيرة واحدة** من الطبق ويتذوقها. هذه **الملعقة** تمثل عينة صغيرة من الطبق كله.

■ **في عالم الشبكات:** الشبكة لا تنظر إلى كل بيانات التدريب (آلاف الصور) دفعة واحدة. هذا مكلف جداً من الناحية الحسابية. بدلاً من ذلك، تختار **مجموعة صغيرة وعشوائية** من البيانات تسمى "mini-batch"، مثلاً 100 صورة وتعمل عليها فقط.

الخطوة 2: حساب التدرجات (Calculating Gradients)

- في عالم الشيف : يتذوق الشيف الملعقة ويقارن طعمها بالطعم المثالي الذي يهدف إليه. ثم يحلل الفرق ويقول لنفسه: أوه، الطعم مالح قليلاً، ويحتاج لمسة فلفل أكثر. هذه النصيحة أو ملاحظات التحسين هي ما نسميه التدرج
- في عالم الشبكات : تأخذ الشبكة الـ "mini-batch" وتقارن مخرجاتها بالنتائج الصحيحة ثم تحسب دالة الخسارة Loss Function التي تخبرها بمدى خطئها. بعد ذلك، تحسب التدرج (Gradient)، وهو عبارة عن متجه يخبر كل مقدار في الوصفة (كل وزن وانحياز) ب:
 - 1- اتجاه التعديل المطلوب (أكثر أم أقل؟).
 - 2- حجم التأثير (هل هذا المقدار هو السبب الرئيسي في الخطأ أم لا؟).

الخطوة 3: تحديث المعاملات (Updating Parameters)

- في عالم الشيف : بناءً على نصيحته (أقل ملح، أكثر فلفل)، يقوم الشيف بتعديل طفيف جداً في الوصفة: يقلل من كمية الملح قليلاً ويضيف رشّة فلفل.
- في عالم الشبكات : تقوم الشبكة بتحديث أوزانها وانحيازاتها في الاتجاه المعاكس للتدرج. إذا كان التدرج يقول هذا الوزن زاد من الخطأ، فإن الشبكة تقلل هذا الوزن قليلاً. هذه الخطوة الصغيرة هي التي تحدث الفرق.

الخطوة 4: التكرار (Repeating)

- في عالم الشيف : لا يكتفي الشيف بتذوق مرة واحدة. يحرك الطبق، ينتظر قليلاً، ثم يأخذ ملعقة أخرى (دفعة صغيرة جديدة) من مكان آخر في الطبق ويكرر (يتذوق، يحلل، يعدّل). يفعل ذلك مراراً وتكراراً.
- في عالم الشبكات : تكرر الشبكة الخطوات من 1 إلى 3 آلاف أو ملايين المرات. في كل مرة، تأخذ "mini-batch" جديد وتحسب التدرجات وتعديل الوصفة قليلاً. مع كل تكرار، تقترب الوصفة (الأوزان والانحيازات) من الكمال.

الخطوة	التشبيه (الشيف الطباخ)	المصطلح التقني
الخطوة 1	أخذ ملعقة صغيرة من الطبق لتذوقها	اختيار "Mini-Batch" عشوائي من البيانات
الخطوة 2	تحليل الفرق بين الطعم الحالي والطعم المثالي.	حساب الخسارة والتدرجات (Gradients)
الخطوة 3	تعديل طفيف في مقادير الوصفة (ملح، فلفل).	تحديث الأوزان والانحيازات (Parameters)
الخطوة 4	تكرار عملية التذوق والتعديل مراراً	تكرار العملية على دفعات جديدة من البيانات

Implementing a Neural Network That Supports Backpropagation

```

175 import numpy as np
176 from collections import OrderedDict
177
178 class TwoLayerNet:
179     def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01):
180         # Initialize weights
181         self.params = {}
182         self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
183         self.params['b1'] = np.zeros(hidden_size)
184         self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
185         self.params['b2'] = np.zeros(output_size)
186         # Create layers
187         self.layers = OrderedDict()
188         self.layers['Affine1'] = Affine(self.params['W1'], self.params['b1'])
189         self.layers['Relu1'] = Relu()
190         self.layers['Affine2'] = Affine(self.params['W2'], self.params['b2'])
191         self.lastLayer = SoftmaxWithLoss()
192
193     def predict(self, x):
194         for layer in self.layers.values():
195             x = layer.forward(x)
196         return x
197         # x: input data, t: label data
198
199     def loss(self, x, t):
200         y = self.predict(x)
201         return self.lastLayer.forward(y, t)

```

```

201 def accuracy (self, x, t):
202     y = self.predict(x)
203     y = np.argmax(y, axis=1)
204     if t.ndim != 1 : t = np.argmax(t, axis=1)
205     accuracy = np.sum(y == t) / float(x.shape[0])
206     return accuracy
207     # x: input data, t: teacher data
208 def gradient(self, x, t):
209     # forward
210     self.loss(x, t)
211     # backward
212     dout = 1
213     dout = self.lastLayer.backward(dout)
214     layers = list(self.layers.values())
215     layers.reverse()
216     for layer in layers:
217         dout = layer.backward(dout)
218     # Settings
219     grads = {}
220     grads['W1'] = self.layers['Affine1'].dw
221     grads['b1'] = self.layers['Affine1'].db
222     grads['W2'] = self.layers['Affine2'].dw
223     grads['b2'] = self.layers['Affine2'].db
224     return grads

```

فكر في هذا الكود على أنه مصنع ومُصمم بشكل مثالي لإنتاج وتحسين الشبكات العصبية.

أولاً: الفكرة الأساسية : مصنع الليغو الذكي

بدلاً من بناء شبكة عصبية عن طريق كتابة كل عملية حسابية (ضرب، جمع، دالة تنشيط) بشكل منفصل، قمنا ببناء وحدات أو قطع ليغو جاهزة مثل Affine, Relu, SoftmaxWithLoss الآن، كل ما علينا فعله هو ترتيب هذه القطع بالترتيب الصحيح، والمصنع سيقوم ببقية العملية تلقائياً.

ثانياً: تفكيك المصنع (TwoLayerNet class)

دعنا نستكشف أقسام هذا المصنع:

القسم 1 : تصميم المصنع وتهيئته (init)

هذه هي مرحلة بناء المصنع وتجهيزه

■ : self.params = {}

هذا هو "المخزن المركزي للمصنع". نخزن فيه أهم الأجزاء القابلة للتعديل: الأوزان (W) والانحيازات (b)

■ : self.layers = OrderedDict()

هذه هي خط التجميع نفسه. نستخدم OrderedDict (قاموس مرتب) لأن الترتيب مهم جداً نضع المحطات بالترتيب:

1- Affine1 : محطة التحويل الخطي الأولى.

2- Relu1 : محطة التنشيط الأولى.

3- Affine2 : محطة التحويل الخطي الثانية.

■ : self.lastLayer = SoftmaxWithLoss()

هذه هي "محطة مراقبة الجودة النهائية هي منفصلة عن خط التجميع لأنها تأتي في النهاية لتقييم المنتج النهائي.

القسم 2 : عملية الإنتاج (predict)

هذه هي عملية تشغيل خط التجميع

■ المدخلات x : هي المادة الخام التي تدخل إلى المصنع.

■ الحلقة for : تأخذ المادة الخام x وتمررها إلى أول محطة (Affine1) تأخذ الناتج من المحطة الأولى وتمرره إلى الثانية (Relu1)، وهكذا حتى النهاية.

■ المخرجات : المنتج النهائي (التنبؤ) y

القسم 3 : قسم مراقبة الجودة loss و accuracy

■ : loss(x, t)

هذه الدالة تقول: شغل المصنع بالكامل للحصول على منتج (predict) ، ثم أرسل هذا المنتج إلى محطة الجودة النهائية (lastLayer) مع المخططات الأصلية (t) لتعطينا تقرير الخسارة (مدى سوء المنتج)

■ : accuracy(x, t)

هذه دالة تقييم أبسط. تقول: شغل المصنع، ثم قارن المنتج النهائي بالمخططات الأصلية وأخبرني بنسبة المنتجات الصحيحة

القسم 4 : حلقة التحسين والتغذية الراجعة (gradient)

هذا هو قلب عملية التعلم، وهو آلية التحسين الذكية في المصنع.

■ : self.loss(x, t)

أولاً، نقوم بتشغيل العملية الأمامية بالكامل. هذه الخطوة ضرورية لأنها تحفظ جميع القيم الوسيطة داخل كل قطعة ليغو، والتي سنحتاجها الآن.

■ : dout = 1

نبدأ إشارة التغذية الراجعة من محطة الجودة النهائية بقيمة أولية.

■ : layers.reverse()

لإصلاح خطأ في المنتج النهائي، هل تبدأ بالفحص من أول محطة في خط التجميع أم من آخر محطة؟ بالطبع من الآخر. هذه الدالة تعكس ترتيب المحطات.

■ : for layer in layers: dout = layer.backward(dout)

نمرر إشارة الخطأ للخلف. كل محطة تستقبل الإشارة، تحسب كيف يجب أن تعدل نفسها (dW, db) ، وتولد إشارة خطأ جديدة للمحطة التي تسبقها.

■ : grads = {...}

الآن، وبعد أن انتهت عملية التغذية الراجعة، نحتاج إلى تجميع تقارير التحسين التي أعدتها كل محطة. نقوم بإنشاء ملف تقارير جديد اسمه grads نذهب إلى محطات التحويل (Affine1, Affine2) وننسخ التقارير التي حفظوها (dW, db) ونضعها في ملفنا الجديد.

النقطة المهمة: هذا الملف (grads) هو الناتج النهائي لهذه الدالة، وهو جاهز ليتم تسليمه إلى إدارة التحديث التي ستقوم بتطبيق هذه التعديلات على المخزن المركزي (self.params) هذه الدالة نفسها لا تقوم بالتحديث، بل فقط بتحضير التعليمات اللازمة للتحديث.

ثالثاً: لماذا OrderedDict هو البطل الخفي؟

- **المشكلة:** القاموس العادي في بايثون لا يضمن الحفاظ على ترتيب العناصر.
- **الحل:** OrderedDict: (القاموس المرتب) يتذكر بالضبط الترتيب الذي أضفت به العناصر.
- **الأهمية:** هذا يضمن أن عملية المرور الأمامي تتم دائماً بالترتيب الصحيح (Affine1 -> Relu1 -> Affine2)، وعملية المرور الخلفي تتم دائماً بالترتيب العكسي الصحيح (Affine2 -> Relu1 -> Affine1).

Gradient Check

لنشرح باستخدام تشبيهاً بسيطاً [سباق بين حاسبة عملاقة سريعة ومعداد يدوي موثوق]

المقدمة: لدينا طريقتان لحساب الانحدار (Gradient)

عندما نريد تدريب شبكة عصبية، نحتاج إلى حساب الانحدار (أو التدرج) لنعرف كيف نعدل الأوزان. هناك طريقتان رئيسية للقيام بذلك:

الطريقة الأولى: الانتشار الخلفي (Backpropagation) الحاسبة العملاقة السريعة

- ما هي؟ هي الطريقة الذكية والمعقدة التي ناقشناها سابقاً. إنها تحسب التدرجات عن طريق تمرير الخطأ للخلف عبر الشبكة طبقة تلو الأخرى.
- **ميزتها:** سريعة وفعالة للغاية! هذا هو ما نستخدمه في التدريب الفعلي للشبكات العملاقة.
- **عيوبها:** معقدة جداً في التنفيذ. لأنها تتضمن الكثير من الخطوات الرياضية والمصفوفات، فمن السهل جداً أن ترتكب خطأ برمجياً بسيطاً يجعل كل الحسابات خاطئة، حتى لو كانت الشبكة تبدو تتعلم.

الطريقة الثانية: التفاضل العددي (Numerical Differentiation) المعداد اليدوي الموثوق

(المعداد اليدوي الموثوق: طريقة حسابية بسيطة جداً وواضحة، مثل طريقة يحسب بها شخص الأشياء يدوياً، خطوة بخطوة، بدلاً من استخدام آلة معقدة)

- ما هي؟ هي طريقة بسيطة جداً ومباشرة لحساب التدرج. فكر فيها كأنها تغير وزناً واحداً بمقدار صغير جداً، وترى كيف تأثرت الخسارة، ثم تكرر هذا لكل وزن على حدة.
- **ميزتها:** بسيطة وموثوقة. من الصعب جداً أن تخطئ في تنفيذها لأن المنطق مباشر جداً.
- **عيوبها:** بطيئة جداً جداً! هي غير عملية على الإطلاق لتدريب الشبكات الحقيقية لأنها ستستغرق وقتاً طويلاً.

المشكلة: كيف نثق في حاسبتنا العملاقة؟

لدينا حاسبة عملاقة سريعة (الانتشار الخلفي) ونريد استخدامها للتدريب، لكننا خائفون من أنها قد تكون معطوبة بسبب تعقيداتها.

كيف يمكننا أن نتأكد من أنها تعطينا أرقامًا صحيحة؟

الحل: فحص الانحدار (Gradient Check) عملية المطابقة

هنا يأتي دور المِعداد اليدوي الموثوق (التفاضل العددي).

فكرة فحص الانحدار بسيطة:

1. نأخذ مشكلة حسابية واحدة (مجموعة واحدة من الأوزان والبيانات).
 2. نعطيها للحاسبة العملاقة السريعة (كود الانتشار الخلفي) ونطلب منها حساب التدرجات. لنفترض أن النتيجة كانت $[0.1, -0.5, 0.2]$
 3. نعطي نفس المشكلة بالضبط للمِعداد اليدوي الموثوق (كود التفاضل العددي) ونطلب منه حساب التدرجات. بما أنه بطيء، سننتظر قليلاً. لنفترض أن النتيجة كانت $[0.101, -0.498, 0.199]$
 4. نقارن النتيجة:
- هل الأرقام متقاربة جداً؟** $[0.1, -0.5, 0.2]$ مقابل $[0.101, -0.498, 0.199]$ نعم، هي متقاربة جداً!
- الاستنتاج:** هذا يعطينا ثقة عالية جداً بأن حاسبتنا العملاقة (كود الانتشار الخلفي) تعمل بشكل صحيح! الفروق الطفيفة هي مجرد تقريب رياضي طبيعي.

خلاصة:

فحص الانحدار (Gradient Check) ليس لتدريب الشبكة، بل هو أداة مطور (Developer Tool)

- **الهدف:** هو التأكد من أن تنفيذنا للانتشار الخلفي (الطريقة السريعة) صحيح وخالي من الأخطاء.
- **الطريقة:** نستخدم التفاضل العددي (الطريقة البطيئة والموثوقة) كـ "إجابة نموذجية" أو "مقياس ذهبي" للمقارنة معه.
- **النتيجة:** إذا تطابقت النتائج، نكون قد نجحنا في التحقق من كودنا. بعد ذلك، يمكننا أن نثق في حاسبتنا العملاقة ونستخدمها للتدريب الفعلي، وننسى أمر المِعداد اليدوي البطيء.

Training Using Backpropagation

```
230 # Load data
231 #
232 network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)
233
234 iters_num = 10000
235 train_size = x_train.shape[0]
236 batch_size = 100
237 learning_rate = 0.1
238
239 train_loss_list = []
240 train_acc_list = []
241 test_acc_list = []
242
243 iter_per_epoch = max(train_size / batch_size, 1)
244
245 for i in range(iters_num):
246     batch_mask = np.random.choice(train_size, batch_size)
247     x_batch = x_train[batch_mask]
248     t_batch = t_train[batch_mask]
249     grad = network.gradient(x_batch, t_batch)
250
251     for key in ('W1', 'b1', 'W2', 'b2'):
252         network.params[key] -= learning_rate * grad[key]
253
254     loss = network.loss(x_batch, t_batch)
255     train_loss_list.append(loss)
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```
257         if i % iter_per_epoch == 0:
258             train_acc = network.accuracy(x_train, t_train)
259             test_acc = network.accuracy(x_test, t_test)
260             train_acc_list.append(train_acc)
261             test_acc_list.append(test_acc)
262             print(train_acc, test_acc)
```

```

-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-3973972994.py in <cell line: 0>()
      3 #
      4
----> 5 network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)
      6
      7 iters_num = 10000

/tmp/ipython-input-3287134202.py in __init__(self, input_size, hidden_size, output_size, weight_init_std)
      9     self.params['b1'] = np.zeros(hidden_size)
     10     self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
--> 11     self.params['b2'] = p.zeros(output_size)
     12     # Create layers
     13     self.layers = OrderedDict()

```

NameError: name 'p' is not defined

```

228 import numpy as np
229 from collections import OrderedDict
230
231 # (يفترض وجود كود الطبقات Affine, Relu, SoftmaxWithLoss هنا كما في التمرين)
232 class ThreeLayerNet:
233     """
234     شبكة عصبية من ثلاث طبقات.
235     Affine - ReLU - Affine - ReLU - Affine - Softmax : بنية الشبكة
236     """
237     def __init__(self, input_size, hidden_size1, hidden_size2, output_size, weight_init_std=0.01):
238         # =====
239         # 1. تهيئة الأوزان والانحيازات (الجزء الأول من التعديل)
240         # =====
241         self.params = {}
242         # الطبقة الأولى
243         self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size1)
244         self.params['b1'] = np.zeros(hidden_size1)
245         # الطبقة الثانية (جديدة)
246         self.params['W2'] = weight_init_std * np.random.randn(hidden_size1, hidden_size2)
247         self.params['b2'] = np.zeros(hidden_size2)
248         # الطبقة الثالثة (كانت الثانية سابقاً)
249         self.params['W3'] = weight_init_std * np.random.randn(hidden_size2, output_size)
250         self.params['b3'] = np.zeros(output_size)
251
252         # =====
253         # 2. بناء خط الإنتاج بالترتيب الصحيح (الجزء الثاني من التعديل)
254         # =====
255         self.layers = OrderedDict()
256         self.layers['Affine1'] = Affine(self.params['W1'], self.params['b1'])
257         self.layers['Relu1'] = Relu()
258         self.layers['Affine2'] = Affine(self.params['W2'], self.params['b2']) # طبقة Affine جديدة
259         self.layers['Relu2'] = Relu() # طبقة ReLU جديدة
260         self.layers['Affine3'] = Affine(self.params['W3'], self.params['b3']) # كانت Affine2 سابقاً
261
262         self.lastLayer = SoftmaxWithLoss()
263
264         # دالة predict لا تحتاج لأي تعديل، لأنها تمر على الطبقات ديناميكياً
265         def predict(self, x):
266             for layer in self.layers.values():
267                 x = layer.forward(x)
268             return x
269
270         # دالة loss لا تحتاج لأي تعديل
271         def loss(self, x, t):
272             y = self.predict(x)
273             return self.lastLayer.forward(y, t)
274

```

```

275 def gradient(self, x, t):
276     # الانتشار الأمامي (لا يحتاج لتعديل)
277     self.loss(x, t)
278
279     # الانتشار الخلفي (لا يحتاج لتعديل في منطقته)
280     dout = 1
281     dout = self.lastLayer.backward(dout)
282
283     layers = list(self.layers.values())
284     layers.reverse()
285     for layer in layers:
286         dout = layer.backward(dout)
287
288     # =====
289     # 3. تجميع التدرجات (الجزء الثالث من التعديل)
290     # =====
291     grads = {}
292     grads['W1'] = self.layers['Affine1'].dW
293     grads['b1'] = self.layers['Affine1'].db
294     grads['W2'] = self.layers['Affine2'].dW # إضافة تدرجات الطبقة الجديدة
295     grads['b2'] = self.layers['Affine2'].db # إضافة تدرجات الطبقة الجديدة
296     grads['W3'] = self.layers['Affine3'].dW
297     grads['b3'] = self.layers['Affine3'].db
298     return grads
299

```

تطبيق عملي لكل المفاهيم التي ناقشناها، وهو يمثل عملية التدريب. في حال لم تكن الصور مفهومة يمكنك رؤية الكود كامل من المحاضرة

دعنا نقسم الشرح إلى ثلاثة أجزاء:

1. شرح الخطأ البرمجي (NameError)
2. شرح كود التدريب خطوة بخطوة.
3. شرح كود ThreeLayerNet الذي قمنا بتعديله .

1. أولاً: إصلاح الخطأ (NameError: name 'p' is not defined)

ماذا يعني؟ يخبرك بايثون: لقد استخدمت متغيراً اسمه p ، لكنني لا أعرف ما هو هذا المتغير! لم يتم تعريفه من قبل أين هو الخطأ ؟ في السطر التالي :

```
self.params['b2'] = p.zeros(output_size)
```

المشكلة هي p.zeros هي بالفعل np.zeros اختصار لـ numpy.zeros

الحل: ببساطة، قم بتغيير p إلى np

الخطأ

```
self.params['b2'] = p.zeros(output_size)
```

التصحيح

```
self.params['b2'] = np.zeros(output_size)
```

2. ثانياً: شرح كود التدريب

هذه الخطوات الأربع لتدريب الشبكة العصبية التي شرحناها سابقاً (اختيار دفعة، حساب التدرجات، تحديث المعاملات، التكرار)

1. بناء الشبكة (network = TwoLayerNet(...)) :

هنا نقوم بإنشاء المصنع الخاص بنا وتحديد مواصفاته (784 مدخل، 50 خلية عصبية مخفية، 10 مخرجات).

2. إعدادات التدريب :

نحدد متغيرات التحكم في عملية التعلم، مثل عدد مرات التكرار وحجم خطوة التحديث (learning_rate)

3. حلقة التدريب (for i in range(iters_num):)

هذه هي الخطوة الرابعة (التكرار). كل دورة في هذه الحلقة تمثل خطوة واحدة من تعلم الشبكة.

4. داخل الحلقة (الخطوات 1، 2، 3):

الخطوة 1 (Mini-Batch)

np.random.choice تختار عشوائياً 100 صورة وتسمياتها من مجموعة التدريب الكبيرة.

الخطوة 2 (Gradients)

network.gradient(...) نطلب من الشبكة أن تأخذ الدفعة الصغيرة (x_batch, t_batch) وتحسب لنا تقارير التحسين (grad) باستخدام الانتشار الخلفي.

الخطوة 3 (Update)

الحلقة (for key in ...) هي التي تقوم بالتحديث الفعلي. تأخذ كل مقدار في المصنع (W1, b1, ...) وتعده قليلاً بناءً على تقرير التحسين الخاص به (grad[key]) ومعدل التعلم.

5. قياس الأداء :

if i % iter_per_epoch == 0 :

هذا الشرط يعني بعد كل دورة كاملة (epoch) على كل بيانات التدريب في هذه النقطة، نقوم بحساب دقة الشبكة على بيانات التدريب وبيانات الاختبار لنرى كيف تتقدم.

ثالثاً: شرح كود ThreeLayerNet

تحويل مصنع من طبقتين إلى مصنع من ثلاث طبقات بنجاح.

لقد قمنا بكل شيء بشكل صحيح:

1. إضافة معاملات جديدة: أضفنا W2, b2 للطبقة الثانية وأعدنا تسمية الطبقة الثالثة لتكون W3, b3.

2. إضافة طبقات جديدة: أضفنا Relu2, Affine2 إلى OrderedDict بالترتيب الصحيح.

3. تجميع التدرجات: قمنا بتحديث دالة gradient لتجمع التقارير من الطبقات الجديدة (Affine2) وتضعها في القاموس grads.

ملاحظة أخيرة :

• الكود الأول (حلقة التدريب) سيعمل مع شبكتنا الجديدة ThreeLayerNet أيضاً،

• كل ما علينا هو تغيير السطر network = TwoLayerNet(...) إلى network = ThreeLayerNet(...)

تحديد الأبعاد الجديدة مثل hidden_size1, hidden_size2

النهاية